C# · .NET 8 · WINDOWS FORMS

Tindahan ni Aling Nena

System architecture, game design, and the four pillars of object-oriented programming

A desktop cashier simulation in which the player runs the counter of a Filipino sari-sari store: read the customer's order, collect payment, and return the exact change from a shared cash drawer.

PROJECT
Tindahan ni Aling Nena (Recoded)

DOCUMENT TYPE
Software design & source-code analysis

SOURCE BASELINE
Commit 8795221 · 23 C# files · 1,456 lines

PREPARED
5 November 2026

Document Information

This document explains how the **Tindahan ni Aling Nena (Recoded)** application is structured, how a round of play works, and how the codebase applies the four pillars of object-oriented programming. It is intended to serve both as a design reference and as an academic project report.

> **HOW TO READ THIS DOCUMENT**
>
> Chapters 1–4 cover the product, its architecture, its domain model, and its rules. Chapter 5 is the dedicated object-oriented analysis requested for this report. Chapters 6–7 assess design quality and the user interface; Chapters 8–11 cover usage, build, testing, and future work. Appendices provide a class-by-class reference, a file map, and a glossary.

Revision and control

Version	Date	Description	Author
1.0	5 Nov 2026	Initial technical documentation based on source-code analysis of commit `8795221`.	Project documentation

Environment snapshot

Item	Value
Language	C# (nullable reference types enabled, implicit usings enabled)
Framework	.NET 8 (`net8.0-windows`), Windows Desktop runtime
UI toolkit	Windows Forms (WinForms)
Entry point	`Program.cs` using top-level statements
Solution	`TindahanNiAlingNena_Recoded.slnx`
Build result	Success — 0 warnings, 0 errors
External dependencies	None beyond the .NET base class library (System.Media, System.Drawing, System.Windows.Forms)

Table of Contents

Page numbers refer to the printed page numbers in the footer of this PDF. Internal entries are clickable in PDF viewers that support document links.

1. Introduction **5**

1.1 Overview 5

1.2 Purpose and Audience 5

1.3 Scope and Method 5

1.4 Key Features 6

1.5 Project Snapshot 6

2. System Architecture **8**

2.1 Architectural Style 8

2.2 Layered Structure 9

2.3 Layer Responsibilities 9

2.4 Dependency Rules 9

2.5 Event-Driven Communication 10

2.6 Runtime Sequence 11

2.7 Project Structure 12

2.8 Component Inventory 12

3. Domain Model **14**

3.1 Domain Overview 14

3.2 Class Diagram 15

3.3 Money Model 15

3.4 Customer Model 16

3.5 Order and Product 17

3.6 Relationships 18

4. Game Mechanics **19**

4.1 Game Loop 19

4.2 Starting Conditions 19

4.3 Order Generation 20

4.4 Customer Generation 20

4.5 Payment Generation 20

4.6 Change Validation and Scoring 22

4.7 Difficulty and Balance 22

5. The Four Pillars of OOP **24**

5.1 Overview 24

5.2 Encapsulation 24

5.3 Abstraction 27

5.4 Inheritance 28

5.5 Polymorphism 29

5.6 How the Pillars Cooperate 31
5.7 Coverage Summary 32
6. Design Principles and Code Quality 33
6.1 SOLID Assessment 33
6.2 Composition and Delegation 33
6.3 Static Factory and Registry 34
6.4 Observer Pattern 34
6.5 Resource Management 35
6.6 Coupling and Cohesion 36
6.7 Observations and Refactoring Roadmap 36
7. User Interface 39
7.1 Screen Composition 39
7.2 Layout and Resizing 39
7.3 Controls and Responsibilities 40
7.4 Event Wiring 40
7.5 Visual System 41
7.6 Accessibility 41
7.7 Assets and Rendering 42
8. User Guide 43
9. Build and Run 45
9.1 Prerequisites 45
9.2 Commands 45
9.3 Project Configuration 46
9.4 Publishing 46
10. Testing and Quality Assurance 48
10.1 Current Test Posture 48
10.2 Manual Test Checklist 48
10.3 Suggested Unit Tests 49
11. Limitations and Future Work 50
11.1 Known Limitations 50
11.2 Improvement Roadmap 51
Appendix A: Class Reference 52
Appendix B: File Map 54
Appendix C: Glossary 55
Appendix D: References 56

CHAPTER 1

1. Introduction

1.1 Overview

Tindahan ni Aling Nena (Recoded) is a single-player, mouse-driven simulation game written in C# with Windows Forms. The player takes the role of the cashier at Aling Nena's neighbourhood *sari-sari* store. Each round presents a customer with a short grocery order and a payment assembled from Philippine bills and coins. The player must return the correct change by drawing notes and coins from a shared cash drawer, then confirm the transaction.

Although the presentation is a game, the core of the program is a small arithmetic and inventory system. Money is modelled as discrete denomination objects rather than floating-point values; the cash drawer is an inventory that grows when customers pay and shrinks when change is handed over; and the round is validated by comparing the value of the selected change against the difference between the payment and the order total. The game therefore provides a compact but genuine domain for demonstrating object-oriented design.

A round has three possible shapes. An *exact-paying* customer hands over the precise total, so no change is needed. An *overpaying* customer hands over more than the total and expects the difference. An *underpaying* customer hands over too little; the player must press **Ask for more** before change can be given. A correct answer increases the score and immediately produces the next customer. An incorrect answer clears the selected change, resets the score to zero, and lets the player try the same customer again. There is no terminal state: the session continues indefinitely, with orders growing as the score rises.

> IN ONE SENTENCE
>
> The program is an endless cashier drill that tests whether the player can compose an exact set of Philippine denominations from a limited shared drawer — built as a layered, event-driven WinForms application around a small object-oriented domain model.

1.2 Purpose and Audience

This document serves three audiences. For **instructors and evaluators**, it explains where and how the four pillars of object-oriented programming appear in the code, with file references and excerpts rather than generic definitions. For **maintainers**, it records the intended architecture, the rules encoded in the services, and the known weak points that should be addressed before extending the program. For **readers new to the repository**, it offers a guided path from the entry point to the individual classes, so that the code can be understood without reverse-engineering it from the debugger.

The document deliberately separates description from evaluation. Chapters 1–4 and 7–9 describe what exists. Chapters 5–6 and 10–11 assess how well it is designed and what should change. This distinction matters because several design choices are reasonable for a student project but would not survive a production review.

1.3 Scope and Method

The analysis covers the complete solution in the repository: the WinForms project, its 23 C# source files, the project file, the solution file, and the runtime assets required to play. Compiler-generated files under `bin/` and `obj/` were excluded, as were the contents of the .NET runtime and WinForms libraries.

Findings were produced by reading every source file, tracing the event wiring from `Program.cs` through `GameSession` and `GameScreen`, reconstructing the rules from the service methods, and performing a clean local build. The build completed with zero warnings and zero errors, which means the design observations in this document are not compiler diagnostics; they are architectural judgements. No automated tests exist in the repository, so runtime behavior was verified by code inspection rather than by executing a test suite.

1.4 Key Features

- **Procedural orders.** Every customer receives one to four products (more as the score grows) drawn from a six-item catalog with Filipino store staples such as *pandesal*, instant noodles, and sardines.
- **Three payment behaviours.** Customer subclasses generate exact, over-, or underpayments, producing three distinct cashier situations from one base contract.
- **Denomination-accurate money.** Six bill values (₱20 to ₱1,000) and three coin values (₱1, ₱5, ₱10) are modelled as immutable-in-practice objects with images and sounds.
- **Shared cash drawer.** A single drawer holds every piece in play, starts with ₱880, receives customer payments, and is decremented when change is handed over. Counts are displayed live on each denomination.
- **Exact-change validation.** The selected change is summed and compared with the overpayment; an error clears the selection and resets the score.
- **Escalating difficulty.** Order size increases with the score, gradually raising the arithmetic load and the number of items on the receipt.
- **Event-driven UI.** The form raises intention events; the session raises state events. Neither side calls the other's methods directly at runtime.
- **Presentation assets.** Customer portraits, bill and coin images, a receipt background, and five WAV sound effects are copied to the output directory by the project file.
- **Zero external packages.** The program depends only on the .NET base class library and WinForms.

1.5 Project Snapshot

Attribute	Value
Solution	`TindahanNiAlingNena_Recoded.slnx` (one project)
Project	`TindahanNiAlingNena_Recoded/TindahanNiAlingNena_Recoded.csproj`
Output type	`WinExe` — a Windows desktop application
Target framework	`net8.0-windows`
Source files	23 C# files in four folders: `Models/`, `Services/`, `UI/`, and the project root
Source size	1,456 physical lines including blanks, comments, and braces
Largest file	`UI/GameScreen.cs` at 272 lines
Assets	6 bill images, 3 coin images, 10 customer portraits, 1 receipt background, 5 sounds
External libraries	None
Persistence	None — score, drawer, and customers exist only in memory
Build status	Success: 0 warnings, 0 errors

Attribute	Value
Automated tests	None in the repository
Repository	`github.com/kvkbkbb/TindahanNiAlingNena_Recoded`

CHAPTER 2

2. System Architecture

2.1 Architectural Style

The application uses a **layered architecture with a passive view**, a variant of the Model–View–Presenter pattern. Four concerns are separated by folders and namespaces: the domain *models*, the *services* that implement rules and generators, the *UI* controls, and a tiny *composition root* in `Program.cs`. A single `GameSession` object holds the mutable game state and acts as the presenter between the rules and the screen.

What makes the design distinctive is that the view is *passive* and the communication is *event-driven in both directions*. The form never reaches into the session to ask for data. Instead, it raises intention events — `DenominationClicked`, `GiveChangeClicked`, `ClearClicked`, `AskForMoreClicked` — and the composition root subscribes the session's methods to them. In the opposite direction, the session raises `StateChanged`, `MessageSent`, and `CustomerStarted`; the composition root subscribes the form's display methods. The two halves therefore know nothing about each other's types, only about delegate signatures.

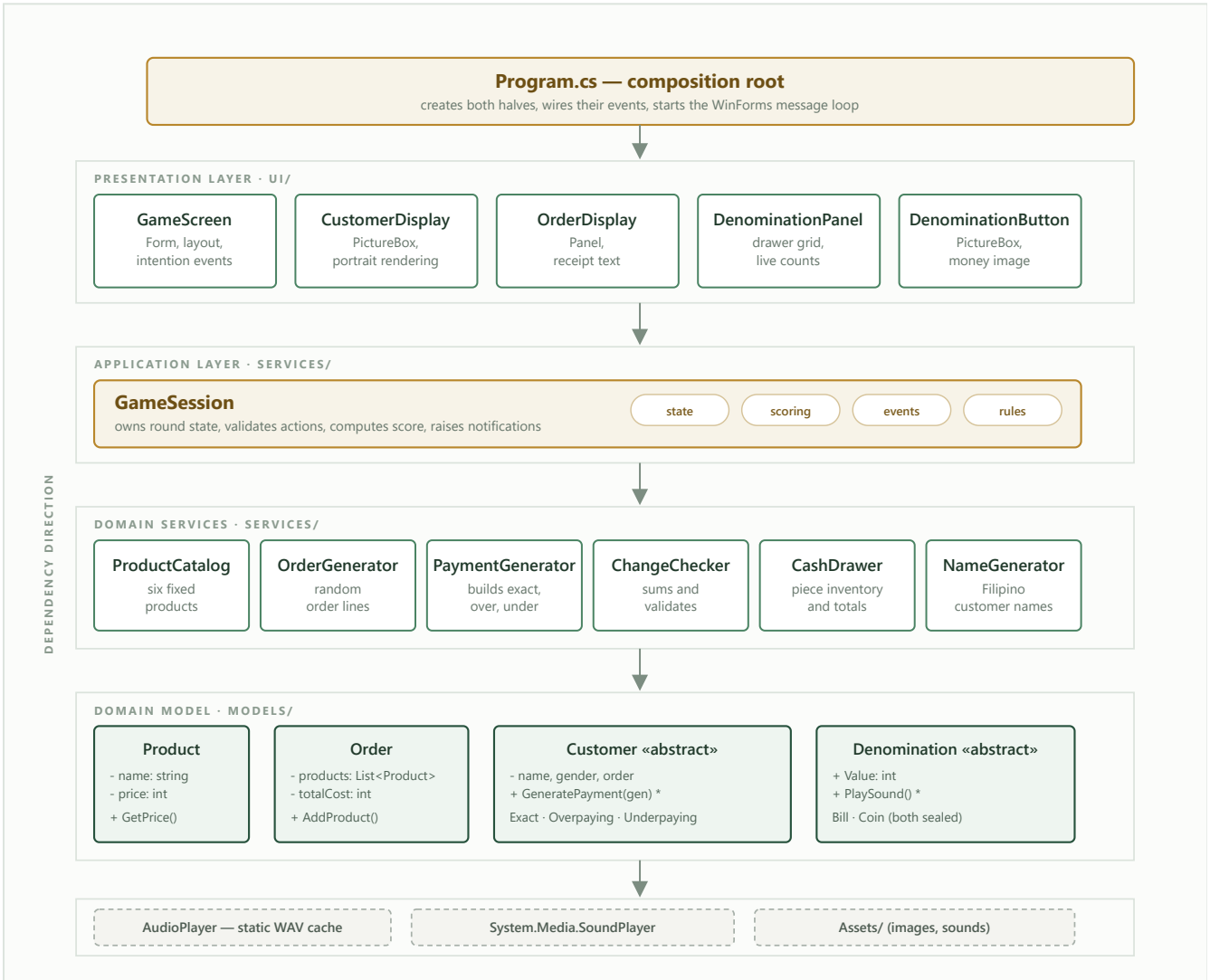


Figure 1. Layered architecture. Solid arrows show compile-time dependency direction. The return path is not a reference: `GameSession` notifies the form through events, so the lower layers never import UI types. `Program.cs` is the only place where both halves are visible.

2.2 Layered Structure

Layer	Location	Contents and purpose
Composition root	`Program.cs`	Top-level statements create the session and the form, connect the eight event handlers, start the first round, and hand control to `Application.Run`. No game logic lives here.
Presentation	`UI/`	Five WinForms types that render state and collect input: the main window, the customer portrait, the receipt, the cash-drawer grid, and the clickable denomination button.
Application	`GameSession` in `Services/`	Owns the current customer, payment, selected change, score, and drawer. Enforces the rules of a turn and publishes state through events.
Domain services	`Services/` (remaining types)	Stateless or narrowly scoped helpers: catalog, order generator, payment generator, change checker, drawer, name generator, and audio playback.
Domain model	`Models/`	Business entities and their behaviour: `Product`, `Order`, the `Customer` hierarchy, and the `Denomination` hierarchy.
Assets	`Assets/`	Bill and coin images, ten customer portraits, a receipt background, and five WAV files. Declared as content in the project file and copied beside the executable.

2.3 Layer Responsibilities

Presentation. `GameScreen` is the window shell. It creates and arranges the four display controls, exposes intention events, and offers three display methods: `ShowState`, `ShowMessage`, and `ShowCustomer`. It contains no scoring, no payment rules, and no drawer bookkeeping. Each child control is equally focused: `CustomerDisplay` paints a portrait, `OrderDisplay` formats receipt text, `DenominationPanel` builds the drawer grid and raises a click event, and `DenominationButton` wraps one denomination image.

Application. `GameSession` is the only stateful service. It answers four questions: what is being sold, who is paying, what has the customer paid, and what change has the player selected. It also owns the score and orchestrates the transition from one customer to the next. The class exposes seven operations in total — four commands (`Start`, `AddChangePiece`, `ClearChange`, `GiveChange`, `AskForMore`) and three events.

Domain services. These are deliberately small. `ProductCatalog` owns the six products. `OrderGenerator` turns a size range into an `Order`. `PaymentGenerator` turns an amount into a list of denominations. `ChangeChecker` answers arithmetic questions. `CashDrawer` is an inventory of physical pieces. `NameGenerator` composes a title and a first name by gender. `AudioPlayer` is infrastructure rather than domain logic, but it lives in the same folder for convenience.

Domain model. The entities own the behaviour that naturally belongs to them: an `Order` knows its products and total, a `Product` knows its name and price, a `Customer` knows how to produce a payment, and a `Denomination` knows its value, image, and sound.

2.4 Dependency Rules

The intended dependency direction is downward: UI depends on services and models, services depend on models, and the composition root depends on everything. That rule holds for the majority of the code. `GameSession` imports only `Models` and `Models.Money`; it never mentions a WinForms type. This is what allows the session to

be exercised without opening a window, at least in principle.

Two upward edges break the pure layering. `Customer.GeneratePayment` takes a `PaymentGenerator`, a service type, so `Models.Customers` depends on `Services`. `Bill.PlaySound` and `Coin.PlaySound` call `AudioPlayer`, so `Models.Money` also depends on `Services`. At the namespace level the dependency is therefore bidirectional. In practice this is harmless in a project of this size, but it means the model cannot be compiled or reused without the service assembly, and it makes the sound effect a hard-wired side effect of a domain object. Chapter 6.7 proposes moving playback behind an interface to restore a single direction.

A second observation is that all dependencies are on concrete classes. There are no interfaces in the repository, and `GameSession` constructs its own collaborators (`new ProductCatalog()`, `new CashDrawer()`, and so on). This simplifies reading but prevents substituting fakes in tests and makes the session responsible for assembling its own dependencies.

2.5 Event-Driven Communication

Events are the contract between the two halves of the program. The form publishes what the player did; the session publishes what changed. The composition root connects matching signatures, so both classes remain unaware of each other.

Event	Signature	Raised by	Handled by
`DenominationClicked`	`Action<Denomination>`	`GameScreen` (re-raised from `DenominationPanel`)	`GameSession.AddChangePiece`
`GiveChangeClicked`	`Action`	`GameScreen`	`GameSession.GiveChange`
`ClearClicked`	`Action`	`GameScreen`	`GameSession.ClearChange`
`AskForMoreClicked`	`Action`	`GameScreen`	`GameSession.AskForMore`
`StateChanged`	`Action<Order, string, int, int, int, IReadOnlyDictionary<int,int>>`	`GameSession`	`GameScreen.ShowState`
`MessageSent`	`Action<string>`	`GameSession`	`GameScreen.ShowMessage`
`CustomerStarted`	`Action<Gender>`	`GameSession`	Lambda in `Program.cs` that calls `ShowCustomer`

All events are synchronous and run on the UI thread. That is appropriate here: handlers only update labels and pictures, so there is no risk of blocking for long. The trade-off is that failures inside a handler propagate straight back into the publisher, and there is no automatic unsubscription. Because the session and the form live for the entire process, the absence of `-=` calls is not a leak in practice, but it would become one if screens were created and destroyed repeatedly.

The state event carries six values rather than an object. It works, but the signature is fragile: adding a seventh value changes every subscriber, and the ordering of two `int` parameters (amount paid and change given) is easy to confuse. A small immutable `GameState` record would make the contract self-documenting. This is discussed again in Chapter 6.

2.6 Runtime Sequence

One complete turn moves through five phases: the player selects money, the session validates the selection, the player confirms, the session checks the arithmetic, and either a new customer is prepared or the score is reset. Figure 2 shows the successful path and the key calls behind it.

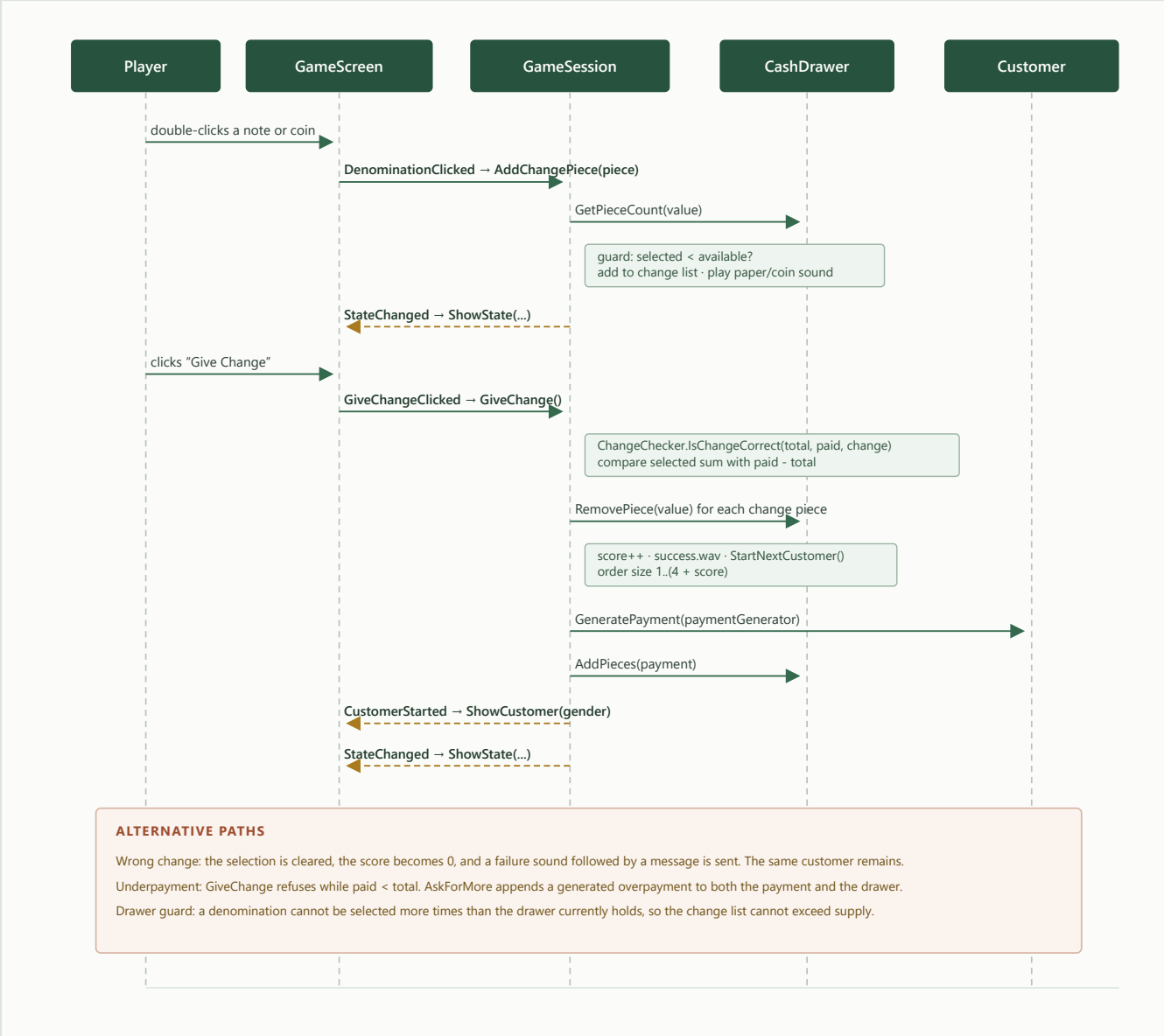


Figure 2. Successful round. Solid arrows are calls; dashed gold arrows are event notifications back to the view. The alternative branches are summarised at the bottom. All calls occur synchronously on the UI thread.

2.7 Project Structure

REPOSITORY LAYOUT (SOURCE ONLY)

```
TindahanNiAlingNena_Recoded.slnx
└── TindahanNiAlingNena_Recoded/
    ├── Program.cs                      entry point + event wiring (21 lines)
    ├── TindahanNiAlingNena_Recoded.csproj
    ├── Models/
    │   ├── Product.cs                  name, price
    │   ├── Order.cs                    product list, running total
    │   ├── Customers/
    │   │   ├── Customer.cs             abstract base + Gender enum
    │   │   ├── ExactPayingCustomer.cs
    │   │   ├── OverpayingCustomer.cs
    │   │   └── UnderpayingCustomer.cs
    │   └── Money/
    │       ├── Denomination.cs         abstract base
    │       ├── Bill.cs                 sealed, 6 bill values
    │       └── Coin.cs                 sealed, 3 coin values
    ├── Services/
    │   ├── GameSession.cs              round state + rules + events
    │   ├── ProductCatalog.cs           fixed product list
    │   ├── OrderGenerator.cs           random order builder
    │   ├── PaymentGenerator.cs         denomination composer
    │   ├── ChangeChecker.cs            sums and validates
    │   ├── CashDrawer.cs               piece inventory
    │   ├── NameGenerator.cs            Filipino name builder
    │   └── AudioPlayer.cs              cached WAV playback
    ├── UI/
    │   ├── GameScreen.cs               main window + layout
    │   ├── CustomerDisplay.cs          portrait control
    │   ├── OrderDisplay.cs             receipt panel
    │   ├── DenominationPanel.cs        drawer grid + counts
    │   └── DenominationButton.cs       clickable money image
    └── Assets/
        ├── Bills/       PHP_20 … PHP_1000 (6 JPGs)
        ├── Coins/       PHP_1, PHP_5, PHP_10 (3 PNGs)
        ├── Customers/   Normal/Men and Normal/Women (10 JPGs)
        ├── Sounds/      button, coin, fail, paper, success (5 WAVs)
        └── Receipt.png  receipt background
```

2.8 Component Inventory

Type	Layer	Responsibility
`Program`	Root	Top-level composition: instantiate, wire, start, run.
`GameScreen`	UI	Window shell; absolute layout of overlays; re-raises four intention events; renders session state.
`CustomerDisplay`	UI	Loads a portrait by customer number and gender; paints it with cover scaling; disposes images.
`OrderDisplay`	UI	Formats order lines, total, and amount paid on a receipt background.

Type	Layer	Responsibility
`DenominationPanel`	UI	Builds a 6-column bill row and a 3-row coin column; shows live counts; raises click event.
`DenominationButton`	UI	Displays one bill or coin; rotates bills 90 degrees; exposes its denomination.
`GameSession`	Application	Owns customer, payment, change, score, and drawer; validates actions; raises three state events.
`ProductCatalog`	Service	Holds six products; returns random product(s).
`OrderGenerator`	Service	Creates an order of random size from the catalog.
`PaymentGenerator`	Service	Builds exact, under-, and overpayments from randomly chosen denominations.
`ChangeChecker`	Service	Totals a list of pieces, counts by value, and validates change.
`CashDrawer`	Service	Starts with ₱880; adds payments; removes change; reports counts and total.
`NameGenerator`	Service	Combines a Filipino title and first name according to gender.
`AudioPlayer`	Infrastructure	Validates a bare filename, caches one `SoundPlayer` per file, plays asynchronously.
`Product`	Model	Immutable-in-practice name and integer price.
`Order`	Model	Collects products and maintains a running total.
`Customer`	Model	Abstract base: order, random gender, generated name, abstract payment method.
`ExactPayingCustomer`	Model	Requests an exact payment.
`OverpayingCustomer`	Model	Requests a payment above the total.
`UnderpayingCustomer`	Model	Requests a payment below the total.
`Denomination`	Model	Abstract base: value, image path, abstract sound.
`Bill`	Model	Sealed paper money; private constructor; static registry of six values; paper sound.
`Coin`	Model	Sealed metal money; private constructor; static registry of three values; coin sound.

Complete type inventory. "Model" types know domain data and behaviour; "Service" types coordinate or compute; "UI" types only present and collect input.

CHAPTER 3

3. Domain Model

3.1 Domain Overview

The domain contains five concepts that matter to the player: a *product*, an *order*, a *customer*, a *denomination*, and the *cash drawer*. Products and orders describe what is being sold. Customers describe who is paying and how much. Denominations describe the physical money. The drawer is the shared inventory from which change is assembled. Everything else — catalogs, generators, checkers, names, audio — exists to create or manipulate those five concepts.

Concept	Implemented by	Meaning in the game
Product	`Product`	A single item on the store shelf, with a display name and an integer peso price.
Order	`Order`	The list of products a customer wants, plus a running total. Duplicates are allowed.
Customer	`Customer` + 3 subclasses	A payer with a name, a gender, an order, and a strategy for producing a payment.
Denomination	`Denomination` + `Bill`/`Coin`	One physical note or coin: a value, an image, and a sound.
Cash drawer	`CashDrawer`	The shared multiset of pieces available for making change and receiving payment.

3.2 Class Diagram

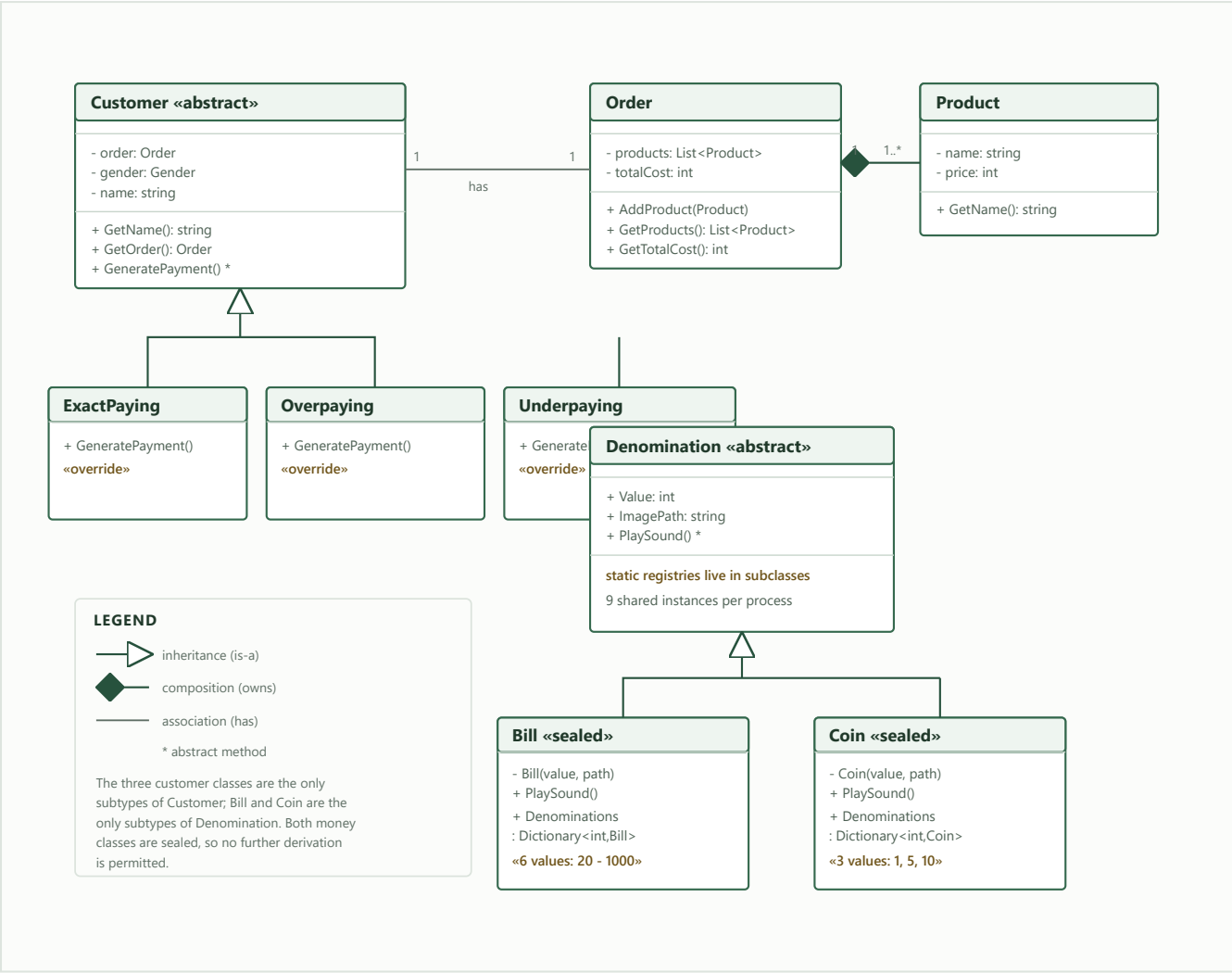


Figure 3. UML class diagram of the domain model. The Customer side uses inheritance for payer behaviour; the Order side uses composition for line items. The money hierarchy is a closed set of nine shared objects. Service collaborators are omitted for readability and described in the component inventory and relationship sections.

3.3 Money Model

Money is represented by an abstract `Denomination` with an integer `Value`, an image path, and an abstract `PlaySound` method. Two sealed subclasses implement the concrete kinds: `Bill` for paper notes and `Coin` for metal pieces. Both use a private constructor and expose a static dictionary keyed by value. The dictionaries are initialized once by the type initializer, so the entire program uses exactly nine denomination objects, regardless of how many pieces are held in the drawer.

Kind	Value	Image	Sound	In starting drawer?
Bill	₱20	`Assets/Bills/PHP_20.jpg`	`paper.wav`	10 pieces
	₱50	`Assets/Bills/PHP_50.jpg`	`paper.wav`	5 pieces
	₱100	`Assets/Bills/PHP_100.jpg`	`paper.wav`	2 pieces
	₱200	`Assets/Bills/PHP_200.jpg`	`paper.wav`	No (comment)
	₱500	`Assets/Bills/PHP_500.jpg`	`paper.wav`	No (comment)
	₱1,000	`Assets/Bills/PHP_1000.jpg`	`paper.wav`	No
Coin	₱1	`Assets/Coins/PHP_1.png`	`coin.wav`	30 pieces
	₱5	`Assets/Coins/PHP_5.png`	`coin.wav`	20 pieces
	₱10	`Assets/Coins/PHP_10.png`	`coin.wav`	10 pieces

Two consequences follow from the static registry. First, it is a **multiton** (a controlled set of named instances): identity can be compared by reference for the nine values, although the program never relies on that. Second, every payment list and every change list contains references to the same objects. That is efficient and conceptually correct for money, whose value is intrinsic, but it also means the objects must be treated as immutable. In this code they are not strictly immutable: `Value` and `ImagePath` have public setters. A caller could change ₱20 into ₱500 for every holder at once. Nothing in the program does so, but the design would be safer with `{ get; private set; }` or constructor-only initialization.

The integer type is the right choice. All prices and payments are whole pesos, and using `int` avoids the rounding errors that would appear if money were stored as `float` or `double` (for example, repeated subtraction in the payment builder). The limitation is that the model cannot represent centavos; that is acceptable for this game, whose smallest unit is the ₱1 coin.

3.4 Customer Model

`Customer` is abstract. Its constructor accepts the order, flips a coin to choose a gender, and asks `NameGenerator` for a matching name. It exposes read-only accessors for the name, gender, and order, and declares a single abstract method:

MODELS/CUSTOMERS/CUSTOMER.CS

```
public abstract class Customer
{
    protected Order order;
    private Gender gender;
    private string name;

    public Customer(Order order)
    {
        this.order = order;
        gender = random.Next(2) == 0 ? Gender.Male : Gender.Female;
        name = NameGenerator.GenerateName(gender);
    }

    public abstract List<Denomination> GeneratePayment(PaymentGenerator paymentGenerator);
}
```

Each subclass overrides only that method, delegating to a different method on the same generator. The result is a very small but genuine application of inheritance and polymorphism: the session can ask any customer for a payment without knowing which payment strategy is in play.

Customer type	Generation call	Resulting payment
`ExactPayingCustomer`	`GenerateExactPayment(total)`	Sum equals the order total; no change is due.
`OverpayingCustomer`	`GenerateOverPayment(total)`	Sum is greater than the total; change is due.
`UnderpayingCustomer`	`GenerateUnderPayment(total)`	Sum is less than the total; the player must ask for more.

Names are assembled from a title and a first name: *Pareng/Kuya/Mang/Tatay/Tito* for men and *Mareng/Ate/Aleng/Nanay/Tita* for women, paired with ten common Filipino first names each. The generator picks uniformly, so there are 50 male and 50 female combinations. The same pool is reused for every customer, which is why names repeat during a long session.

The `Gender` enum is declared in the global namespace rather than inside `Models.Customers`. It is therefore visible everywhere without a `using` directive, including in the UI layer. This works, but it leaks a domain detail into the global scope and is inconsistent with every other type in the project.

3.5 Order and Product

`Product` is a tiny value-like class with two private fields and two getters. It has no setters, so a product cannot be repriced after construction. `Order` composes a list of products and maintains `totalCost` incrementally inside `AddProduct`. That makes `GetTotalCost` an O(1) lookup instead of a repeated sum, at the cost of having two sources of truth that must stay consistent. Because `AddProduct` is the only mutation point, they do.

Product	Price	Notes
Candy	₱1	Smallest possible order; guarantees the payment builder can always finish.
Pandesal	₱2	Cheap staple.
Sachet Shampoo	₱6	Mid-range sachet.

Product	Price	Notes
Instant Noodles	₱12	Common order item.
Coke Sakto	₱15	Beverage.
Ligo Sardines	₱25	Most expensive product.

Products are selected with replacement, so an order can contain the same product more than once. The receipt prints one line per item rather than grouping quantities, which is honest but makes a large order visually repetitive. The order total is always an integer, and there is no tax, discount, or quantity field.

3.6 Relationships

Relationship	UML kind	Multiplicity and meaning
`Customer` → `Order`	Association	1 to 1. Every customer is constructed with exactly one order and exposes it through `GetOrder`.
`Order` → `Product`	Composition	1 to 0..*. The order owns its line list; products are shared catalog objects in practice, but the order controls the collection.
`Customer` → subclasses	Inheritance	Generalisation. Three concrete strategies for one abstract contract.
`Denomination` → `Bill/Coin`	Inheritance	Generalisation into sealed specialisations; no further subtypes are possible.
`CashDrawer` → `Denomination`	Aggregation	0..* to 0..*. The drawer holds references to shared pieces; it does not own their lifetime.
`GameSession` → services	Composition	One session owns one catalog, generator, drawer, checker, and current customer for its lifetime.
`Customer` → `PaymentGenerator`	Dependency	A generator is passed as a method parameter; the customer does not store it.
`Bill/Coin` → `AudioPlayer`	Dependency	A static call from `PlaySound`; no object reference is retained.

> **DESIGN NOTE**
>
> The drawer is an *aggregation*, not a composition, because the note and coin objects pre-exist in the static registries and are shared by every list in the program. Removing a piece from the drawer does not destroy it; it simply drops one reference. That distinction is visible in the code: `CashDrawer` never calls `new Bill(...)` or `new Coin(...)`; it only reuses the registry instances.

CHAPTER 4

4. Game Mechanics

4.1 Game Loop

The program has no explicit state-machine class; the loop is encoded in the methods of `GameSession`. The implicit states are:

1. **Ready.** A customer exists, the payment has been placed in the drawer, and the selected change list is empty.
2. **Selecting change.** The player adds pieces one by one. Each addition is checked against the drawer and re-renders the state. The state persists until the player confirms or clears.
3. **Validation.** `GiveChange` compares the selected sum with the required change and either accepts or rejects the attempt.
4. **Transition.** On success, the score increases and a new customer is built immediately. On failure, the selection is cleared and the score resets to zero; the same customer remains.

Underpayment adds a sub-state: while `paid < total`, giving change is refused and the only productive action is **Ask for more**, which appends money until the payment covers the order. There is no timer, no move limit, and no game-over state.

4.2 Starting Conditions

`CashDrawer` is populated in its constructor. The quantities deliberately favour small denominations, because those are the pieces used for change:

Denomination	Starting count	Subtotal	Cumulative
₱1 coin	30	₱30	₱30
₱5 coin	20	₱100	₱130
₱10 coin	10	₱100	₱230
₱20 bill	10	₱200	₱430
₱50 bill	5	₱250	₱680
₱100 bill	2	₱200	₱880
Total	**77 pieces**	**₱880**	

The ₱200, ₱500, and ₱1,000 entries are commented out, so those notes are not stocked at the start. They can still enter the drawer through payments because `PaymentGenerator` draws from the full bill registry, including high values. Once a high bill is in the drawer, it can be given as change; there is no mechanism to exchange a large note for smaller ones.

4.3 Order Generation

On every new customer, `StartNextCustomer` calls `orderGenerator.GenerateOrder(1, 4 + score)`. `OrderGenerator` chooses a count uniformly in that inclusive range and repeatedly asks the catalog for a random product. With a score of zero, the order contains one to four items. At a score of ten it contains one to fourteen. Because the catalog is uniform, the expected price of one line is ₱10.17 (the mean of 1, 2, 6, 12, 15, and 25), so the expected initial order is about ₱25.42 for 2.5 items.

Score	Item range	Maximum order value	Typical effect
0	1–4	₱100	Short, quick rounds.
1	1–5	₱125	Barely noticeable change.
5	1–9	₱225	Receipt begins to scroll.
10	1–14	₱350	Arithmetic becomes the main challenge.
25	1–29	₱725	Large payments, more change.
50	1–54	₱1,350	High bills appear frequently.

The growth is linear and unbounded. Order size is the only difficulty lever; customer mix, drawer composition, and time pressure never change. This makes the game harder only in the sense of "more sums to keep track of", not in the sense of new decisions.

4.4 Customer Generation

After the order is generated, `StartNextCustomer` draws an integer from 0 to 99 and maps it to a customer type:

Draw	Probability	Customer created	Payment relative to total
0–9	10%	`ExactPayingCustomer`	Equal
10–79	70%	`OverpayingCustomer`	Greater
80–99	20%	`UnderpayingCustomer`	Less

Gender is an independent 50/50 coin flip in the `Customer` constructor. The name is generated from the gender, and `CustomerStarted` passes the gender to the view so that a matching portrait is chosen from the men's or women's folder. Customer type affects only the initial payment; once created, an underpaying customer does not change type even after **Ask for more** makes the payment sufficient.

4.5 Payment Generation

`PaymentGenerator` flattens the bill and coin registries into one list sorted from smallest to largest. Its central routine, `BuildPayment`, repeatedly collects the denominations that still fit the remaining amount, picks one uniformly at random, appends it, and subtracts its value. The loop always terminates because the ₱1 coin fits every positive remainder.

SERVICES/PAYMENTGENERATOR.CS — CORE ALGORITHM

```
private List<Denomination> BuildPayment(int targetAmount)
{
    var paymentDenominations = new List<Denomination>();
    int amountLeftToCover = targetAmount;

    while (amountLeftToCover > 0)
    {
        var denominationsThatStillFit = new List<Denomination>();

        foreach (var denomination in _allDenominations)
        {
            if (denomination.Value <= amountLeftToCover)
                denominationsThatStillFit.Add(denomination);
        }

        int randomIndex = _random.Next(denominationsThatStillFit.Count);
        var chosenPiece = denominationsThatStillFit[randomIndex];

        paymentDenominations.Add(chosenPiece);
        amountLeftToCover -= chosenPiece.Value;
    }

    return paymentDenominations;
}
```

The three public methods wrap this routine with different targets:

Method	Target logic	Guarantee
`GenerateExactPayment(amount)`	Build exactly `amount`.	Sum = amount.
`GenerateUnderPayment(amount)`	Pick a random target in `[1, amount-1]`; for `amount = 1`, return an empty list.	Sum < amount.
`GenerateOverPayment(amount)`	Choose `maxOver = max(10, amount / 2)`; pick a random extra in `[1, maxOver]`; target = `amount + extra`.	Sum > amount.

All three reject non-positive inputs with `ArgumentOutOfRangeException`. The random choice among fitting denominations biases the output toward smaller pieces: for a ₱25 target, the fitting set contains 1, 5, 10, and 20, so a ₱20 note is chosen only one time in four; once the remainder falls below 20, only smaller pieces remain. Consequently, generated payments contain many coins, and large bills appear mainly when the target itself is large. This is plausible cashier behaviour, but it is not modelled on how real customers choose notes.

SUBTLE BEHAVIOUR

`AskForMore` calls `GenerateOverPayment(orderTotal - amountPaid)`. The argument is the *shortfall*, not the total, so the generated extra is relative to the shortfall. For example, if the order is ₱100 and the customer paid ₱80, the generator targets ₱20 plus up to ₱10 extra, producing a payment between ₱101 and ₱110 in total. The customer therefore ends up overpaying, and change is required. This is likely intentional — it creates a change-giving step — but the method name alone does not communicate it.

4.6 Change Validation and Scoring

`ChangeChecker` is deliberately arithmetic-only. `GetTotal` sums a list, `CountPieces` counts occurrences of a value, and `IsChangeCorrect` returns `GetTotal(changeGiven) == amountPaid - orderTotal`. There is no notion of an "optimal" or "expected" set of denominations: any combination that sums to the overpayment is accepted, even if it empties the drawer of coins unnecessarily.

Action	Precondition	Outcome
Double-click a denomination	The number already selected of that value is below the drawer count.	The piece is appended to the change list, its sound plays, and the state refreshes.
Double-click beyond supply	The selected count equals the drawer count.	A message is shown; nothing is added.
Clear	Any state.	The selected change list is emptied and the display refreshes.
Give Change	No current customer.	Silent no-op.
Give Change	Amount paid is less than the order total.	Failure sound and "The customer didn't pay enough! Ask for more money." The score is unchanged.
Give Change	Selected sum does not equal `paid - total`.	Selection cleared, score reset to 0, failure sound, "Wrong change! Try again."
Give Change	Selected sum equals the required change.	Each selected piece is removed from the drawer; score +1; success sound; next customer.
Ask for more	No current customer.	Silent no-op.
Ask for more	Amount paid already covers the total.	Failure sound and "The customer already paid enough!"
Ask for more	Amount paid is short.	An overpayment of the shortfall is generated, appended to the payment, and placed in the drawer; the display refreshes.

The scoring rule is unusually harsh: a single wrong attempt erases the entire streak, even if the player had answered twenty customers correctly. There is no partial credit and no persistent high score. Conversely, the game cannot be lost, so the score is the only measure of performance.

One guard is worth highlighting. `AddChangePiece` compares the number of already-selected pieces of a value against `cashDrawer.GetPieceCount(value)`. Because the player's selected pieces are still physically in the drawer until `GiveChange` succeeds, the guard prevents the selection from exceeding supply. `SendState` then presents the drawer counts *net of the selection*, so the number shown on each button is what remains available if the player confirms immediately.

4.7 Difficulty and Balance

The economy is conservative in total value but fragile in denomination mix. On every successful round the drawer gains the customer's payment and loses the player's change. Since `payment - change = order total`, the drawer's total value rises by the order total each round. Nothing removes money from the system. After a long session the drawer can be worth many thousands of pesos.

However, total value is not the same as usable change. The drawer can accumulate large bills while running short of ₱1, ₱5, and ₱10 coins, because `BuildPayment` favours small pieces and players may spend coins to satisfy exact sums. If the coins run out, the player may be unable to form the required amount even though the drawer is rich in notes, and there is no exchange or "break a bill" action. The game does not detect this dead end. In practice it is rare early on, because the starting drawer contains 60 small coins and every customer payment replenishes the pool, but it becomes possible in long sessions. A future version should either add a bill-breaking action or track a "can make change" invariant and restock when it is violated.

Other balance observations:

- **No time pressure.** The player can calculate indefinitely; the challenge is arithmetic accuracy, not speed.
- **No loss condition.** The session always continues, which makes the game a drill rather than a survival challenge.
- **Reset-only penalty.** Wrong change is punished by losing the streak but not by losing money or customers.
- **Fixed customer mix.** The 10/70/20 split never changes, so later rounds are not more varied, only larger.
- **Order size cap.** There is no cap; at very high scores the receipt can contain dozens of lines and the order value can exceed ₱1,000, which the payment generator can still represent because the ₱1,000 bill is registered.

CHAPTER 5

5. The Four Pillars of OOP

5.1 Overview

Object-oriented programming rests on four ideas: **encapsulation** (protect state behind a controlled interface), **abstraction** (model the essential and hide the incidental), **inheritance** (derive specialised types from a general one), and **polymorphism** (let one call site produce different behaviour depending on the runtime type). The program applies all four, but not to the same degree. Encapsulation and inheritance are pervasive; abstraction is concentrated in two abstract base classes; polymorphism is genuine but limited to the two domain hierarchies, the event delegates, and the WinForms overrides.

The table below is the short version. The rest of the chapter supports each claim with code evidence and then notes where the implementation falls short of the ideal.

Pillar	Primary evidence	Assessment
Encapsulation	`Product`, `Order`, `CashDrawer`, `GameSession`, private `Bill`/`Coin` constructors	Strong inside the classes; several getters leak mutable internals, and two money properties have public setters.
Abstraction	`abstract Customer`, `abstract Denomination`, the `GameSession` command surface	Clear and purposeful, but the abstract contracts say little about what implementations must guarantee.
Inheritance	Three customer subclasses; `Bill`/`Coin`; five UI types deriving from WinForms controls	Shallow (depth two), well motivated, and closed with `sealed` where appropriate.
Polymorphism	`Customer.GeneratePayment`, `Denomination.PlaySound`, overridden `OnPaint`/`Dispose`, event delegates	Real runtime dispatch in daily use; one type test (`is Bill`) and one gender switch reveal places where polymorphism could replace conditionals.

Supporting language mechanics

The pillars are supported by ordinary C# features that are worth naming in a design review: classes and objects, constructors (including `base(order)` chaining), fields versus properties, access modifiers, `static` members, `readonly` fields, constants, enums, delegates and events, generics (`List<T>`, `Dictionary<TKey,TValue>`), nullable reference types, pattern matching, and `IDisposable`. The project uses all of these; it does **not** use interfaces, records, generics of its own, operator overloading, indexers, or user-defined exceptions.

5.2 Encapsulation

Encapsulation is the bundling of state with the operations that maintain it, plus the restriction of direct access from outside. The clearest examples are the small data-holding classes. `Product` stores `name` and `price` as private fields and exposes only getters; `Order` owns its product list and running total and exposes `AddProduct`, `GetProducts`, `GetProductCount`, and `GetTotalCost`.

MODELS/PRODUCT.CS AND MODELS/ORDER.CS — STATE BEHIND BEHAVIOUR

```
public class Product
{
    private string name;
    private int price;

    public Product(string name, int price)
    {
        this.name = name;
        this.price = price;
    }

    public string GetName()
    {
        return name;
    }

    public int GetPrice()
    {
        return price;
    }
}

public class Order
{
    private List<Product> products = new List<Product>();
    private int totalCost = 0;

    public void AddProduct(Product product)
    {
        products.Add(product);
        totalCost += product.GetPrice();   // the invariant lives in one place
    }

    public int GetTotalCost()
    {
        return totalCost;
    }
}
```

The invariant "the total equals the sum of the prices" is preserved because `totalCost` is private and can only change through `AddProduct`. A caller cannot set the total directly. The same pattern appears in `CashDrawer`, whose `pieces` list is private and whose only mutations are `AddPiece`, `AddPieces`, and `RemovePiece`. `GameSession` goes further: its customer, payment, change, score, and drawer fields are all private, and the outside world can interact only through five public methods and three events.

Encapsulation at the type boundary

`Bill` and `Coin` use private constructors. Code outside the class cannot create an arbitrary note or coin; it must go through the static `Denominations` dictionary, which contains exactly the legal values. This protects a real invariant: there can never be a ₱37 bill. The constructors are also the only place where `Value` and `ImagePath` are assigned.

MODELS/MONEY/BILL.CS — CONTROLLED CONSTRUCTION AND A CLOSED VALUE SET

```
public sealed class Bill : Denomination
{
    private Bill(int value, string imagePath)
    {
        Value = value;
        ImagePath = imagePath;
    }

    public override void PlaySound() => AudioPlayer.PlaySound("paper.wav");

    public static Dictionary<int, Bill> Denominations { get; } = new()
    {
        [20]   = new Bill(20,   Path.Combine("Assets", "Bills", "PHP_20.jpg")),
        [50]   = new Bill(50,   Path.Combine("Assets", "Bills", "PHP_50.jpg")),
        [100]  = new Bill(100,  Path.Combine("Assets", "Bills", "PHP_100.jpg")),
        [200]  = new Bill(200,  Path.Combine("Assets", "Bills", "PHP_200.jpg")),
        [500]  = new Bill(500,  Path.Combine("Assets", "Bills", "PHP_500.jpg")),
        [1000] = new Bill(1000, Path.Combine("Assets", "Bills", "PHP_1000.jpg"))
    };
}
```

Events are also encapsulated. In C#, an event declared as `public event Action<string>? MessageSent;` can be subscribed to and unsubscribed from by any code, but it can only be *raised* from inside the declaring class. `GameScreen` therefore cannot fake a session message, and no external type can force `StateChanged` to fire. The invocation list is hidden; only the publisher decides when to notify.

Access modifiers in use

Modifier	Representative use	Purpose
`private`	`Product.name`, `Order.totalCost`, `CashDrawer.pieces`, `GameSession.score`	Hide state that must only change through class methods.
`protected`	`Customer.order`	Let the three subclasses read the order without exposing it publicly.
`public`	`AddChangePiece`, `GeneratePayment`, `GetTotalCost`, events	Define the contract used by other layers.
`readonly`	`PaymentGenerator._random`, `_allDenominations`; `AudioPlayer.Players`	Prevent replacing the reference after construction (the object itself may still be mutable).
`const`	`CustomerDisplay.TotalCustomers`, `GameScreen.HeaderHeight`, `DrawerHeight`	Name fixed values instead of scattering magic numbers.
`static`	`Bill.Denominations`, `Coin.Denominations`, `AudioPlayer`, `NameGenerator`	Share one registry or utility per process rather than per object.

Where encapsulation leaks

- `Order.GetProducts()` returns the internal `List<Product>`. A caller can `Add` or `Remove` items without updating `totalCost`, breaking the class invariant. Returning `IReadOnlyList<Product>` or a copy would

close the leak.

- `Customer.GetOrder()` returns the mutable `Order`. This is currently harmless because only the session reads it, but the door is open.
- `Denomination.Value` and `ImagePath` have public setters. Since all nine objects are shared, mutating one would change it everywhere. Prefer `{ get; private set; }` or constructor-only fields.
- `Bill.Denominations` exposes a mutable `Dictionary`. External code could add, remove, or replace entries. A read-only wrapper would preserve the closed set.
- `Customer.order` is `protected` rather than private, so subclasses could reassign the field. They currently only read it.

These are not functional bugs today, because the program is small and disciplined about access. They are, however, exactly the kind of boundary weakness an evaluator should be able to identify: encapsulation is not just "fields are private", it is "no path exists to an inconsistent state".

5.3 Abstraction

Abstraction means describing *what* a thing does and hiding *how* it does it. The project has two textbook examples, both abstract classes:

Abstraction	Hides	Exposes
`Customer`	How gender and name are chosen; which payment strategy is used; how the order was built.	A name, a gender, an order, and the promise to `GeneratePayment`.
`Denomination`	Whether the piece is paper or metal; the image file; the sound file; the numeric value.	`Value`, `ImagePath`, and the promise to `PlaySound`.

MODELS/CUSTOMERS/CUSTOMER.CS AND AN IMPLEMENTATION

```
public abstract class Customer
{
    protected Order order;
    private Gender gender;
    private string name;

    public Customer(Order order) { /* random gender, generated name */ }

    public abstract List<Denomination> GeneratePayment(PaymentGenerator paymentGenerator);
}

public class OverpayingCustomer : Customer
{
    public OverpayingCustomer(Order order) : base(order) { }

    public override List<Denomination> GeneratePayment(PaymentGenerator paymentGenerator)
        => paymentGenerator.GenerateOverPayment(order.GetTotalCost());
}
```

The base class defines a contract and supplies the shared parts (order, gender, name). The subclass supplies only the variable part. A caller that holds a `Customer` reference does not need to know whether the payment will be exact, over, or under; it only needs to know that a list of denominations will come back. That is the essence of abstraction: a stable interface over varying implementations.

`GameSession` is a second kind of abstraction, even though it is a concrete class. Its public surface — `Start`, `AddChangePiece`, `ClearChange`, `GiveChange`, `AskForMore`, and three events — hides the entire machinery of orders, payments, drawer bookkeeping, and scoring. The UI does not know that a `ChangeChecker` exists, nor that the drawer is a list of denomination references. It knows only the commands and the state notification.

The UI layer also relies on abstraction through the WinForms base classes. `CustomerDisplay` does not reimplement mouse handling, layout, or painting infrastructure; it inherits it and overrides only the two hooks it needs. `OrderDisplay` inherits scrolling and background painting from `Panel`. This is framework-level abstraction: the base class defines the template, the subclass fills in the specific behaviour.

Weaknesses of the abstractions

- `Customer.GeneratePayment` has no documented postcondition. The base class does not say that the returned sum must equal, exceed, or fall short of the total; those relationships exist only in the three implementations and in comments in `PaymentGenerator`. A stronger contract would be visible to callers and testable.
- `Customer` hard-codes random gender selection in its constructor, so a caller cannot create a customer with a known gender for testing. The abstraction hides too much: identity generation is not a customer responsibility.
- `Denomination.PlaySound` hard-wires audio playback into the domain object. A cleaner abstraction would let the domain describe the *kind* of sound and let infrastructure play it.
- There are no interfaces. A `ChangeChecker` could be an `IChangeChecker`, a `CashDrawer` an `IDrawer`, and `AudioPlayer` an `ISoundPlayer`, allowing substitutes in tests. The absence is acceptable for a 1,456-line program but limits testability and extension.

5.4 Inheritance

Inheritance creates an "is-a" relationship. The project uses it deliberately in three places: the customer hierarchy, the money hierarchy, and the UI hierarchy imposed by WinForms.

Base class	Derived class	Inherited	Added or overridden
`Customer` (abstract)	`ExactPayingCustomer`	order, gender, name, getters, constructor	`GeneratePayment` → exact
	`OverpayingCustomer`	order, gender, name, getters, constructor	`GeneratePayment` → over
	`UnderpayingCustomer`	order, gender, name, getters, constructor	`GeneratePayment` → under
`Denomination` (abstract)	`Bill` (sealed)	Value, ImagePath	`PlaySound` → paper; static registry
	`Coin` (sealed)	Value, ImagePath	`PlaySound` → coin; static registry
`Form`	`GameScreen`	window, message loop, events, layout	composition of controls, layout logic, display API

Base class	Derived class	Inherited	Added or overridden
`PictureBox`	`CustomerDisplay`	image surface, sizing, painting	portrait loading, `OnPaint`, `Dispose`
`Panel`	`OrderDisplay`	scroll container, background	receipt formatting
`TableLayoutPanel`	`DenominationPanel`	grid layout, table cells	drawer construction, click event, counts
`PictureBox`	`DenominationButton`	image surface, mouse events	denomination reference, tooltip, rotation

All three domain subclasses call `: base(order)`, so the shared initialisation runs exactly once and cannot drift between implementations. `Bill` and `Coin` are marked `sealed`, which prevents a third level of derivation. That is a good decision: there is no meaningful subtype of a twenty-peso bill, and sealing protects the static registry from unauthorised variants.

The inheritance depth is only two in every case. Shallow hierarchies are easy to reason about and avoid the fragile-base-class problem. The UI inheritance is not optional: WinForms requires deriving from `Form`, `Panel`, or `PictureBox` to participate in the framework. In the domain, inheritance is optional but well justified, because the subclasses differ only in behaviour and share all state.

> **INHERITANCE VERSUS COMPOSITION**
>
> The program also uses composition where inheritance would be wrong. `GameSession` does not inherit from `CashDrawer`; it *has* a drawer. `Customer` does not inherit from `Order`; it *has* an order. `GameScreen` does not inherit from `CustomerDisplay`; it contains one. The rule of thumb applied here is consistent: inherit only when the subtype passes the "is-a" test and genuinely reuses the base contract.

The main limitation is that customer variation is modelled as a class hierarchy when it could be a strategy object. Adding a "customer who pays with a rounded-up amount" requires a new class *and* a change to the factory logic in `GameSession`. A single `Customer` class holding an `IPaymentStrategy` would allow new behaviours without touching the factory. For three behaviours this is a matter of taste; the current design is more readable for a beginner audience.

5.5 Polymorphism

Polymorphism lets the same call expression run different code depending on the object's runtime type. In this program it appears in four forms.

1. Overridden domain methods

The most important example is the customer payment method. `GameSession` stores the current customer as the abstract base type and calls `GeneratePayment` without a type test:

SERVICES/GAMESESSION.CS — ONE CALL, THREE BEHAVIOURS

```
private Customer? currentCustomer;
// ...
currentCustomer = new ExactPayingCustomer(order);      // or Overpaying / Underpaying
// ...
currentPayment = currentCustomer.GeneratePayment(paymentGenerator);
```

Call site	Declared type	Possible runtime types	Behaviour selected
`currentCustomer.GeneratePayment(...)`	`Customer`	Exact, Overpaying, Underpaying	Exact, over-, or underpayment
`piece.PlaySound()`	`Denomination`	`Bill`, `Coin`	`paper.wav` or `coin.wav`
`customerDisplay.OnPaint(e)`	`PictureBox` (framework)	`CustomerDisplay`	Cover-scaled portrait drawing
`customerDisplay.Dispose(true)`	`PictureBox` (framework)	`CustomerDisplay`	Dispose the cached image, then base cleanup

2. Polymorphic collections

`CashDrawer`, `PaymentGenerator`, and `ChangeChecker` all work with `List<Denomination>`. The list can contain any mixture of bills and coins, and operations such as `piece.Value` or `piece.PlaySound()` work without knowing the concrete kind. When the drawer removes a piece, it searches by value and calls `RemoveAt`; it never asks whether the piece is paper or metal. This is subtype polymorphism applied to a collection — the canonical reason to have a base class in the first place.

3. Delegates and events

A delegate is a type-safe reference to a method. The event wiring in `Program.cs` is polymorphism of behaviour: the same event invocation can call different methods depending on what was subscribed.

PROGRAM.CS — DELEGATES CONNECT THE TWO HALVES

```
screen.DenominationClicked += session.AddChangePiece;
screen.GiveChangeClicked  += session.GiveChange;
screen.ClearClicked       += session.ClearChange;
screen.AskForMoreClicked  += session.AskForMore;

session.StateChanged    += screen.ShowState;
session.MessageSent     += screen.ShowMessage;
session.CustomerStarted += gender => screen.ShowCustomer(CustomerState.Normal, gender: gender);
```

At the call site inside `GameScreen`, `DenominationClicked?.Invoke(piece)` does not know that `AddChangePiece` exists. The delegate holds the method reference and dispatches it at runtime. The same mechanism allows multiple subscribers; the program happens to use one per event. The lambda for `CustomerStarted` is a small adapter that fixes one argument (`CustomerState.Normal`) and forwards the gender.

4. Framework overrides

WinForms calls virtual methods on the base control at well-defined times. `CustomerDisplay` overrides `OnPaint` to draw the portrait with cover scaling, and `Dispose(bool)` to release the cached image. The framework holds a `PictureBox` reference but executes the derived implementation. This is the template-method pattern in practice: the base class defines the algorithm, the subclass fills in a step.

Where polymorphism is replaced by conditionals

Two places bypass runtime dispatch and inspect type or category instead:

- `DenominationButton` asks `if (denomination is Bill)` to decide whether to rotate the image. A virtual property such as `bool IsPortrait` or a `DisplayRotation` on `Denomination` would let the object decide, removing the type test. This is the classic "switch on type" smell.
- `CustomerDisplay.GetRandomCustomerNumber` uses a `switch` expression on `Gender?`, and `NameGenerator.GenerateName` uses an `if/else` on `Gender`. These are simple two-way data selections, so a conditional is defensible; a lookup table or a small gender-profile object would be more extensible.

There is no method overloading or operator overloading in the project, so compile-time polymorphism is not part of the design. All interesting dispatch is runtime dispatch through virtual methods or delegates.

> **LISKOV SUBSTITUTION**
>
> The three customer subclasses are substitutable in the narrow sense that all return a valid `List<Denomination>`. They are not substitutable under an implicit "payment covers the total" contract, because `UnderpayingCustomer` deliberately violates it. The program works because the caller handles the underpaid case explicitly. This is a good discussion point: the base contract is intentionally weak, and the subtype's behaviour is a *different situation*, not a violation of a stated rule.

5.6 How the Pillars Cooperate

The pillars are easiest to see when tracing a single interaction. Consider the player double-clicking a ₱20 bill:

1. **Encapsulation:** the button does not expose the drawer or the session; it only exposes `GetDenomination()`. The session's state remains private.
2. **Abstraction:** the button holds a `Denomination`, not a `Bill`. The session accepts a `Denomination`. Neither side depends on the concrete money type.
3. **Polymorphism:** `piece.PlaySound()` dispatches to `Bill.PlaySound` and plays `paper.wav`; the same line would play `coin.wav` for a coin.
4. **Encapsulation again:** `CashDrawer.GetPieceCount` checks supply without revealing the internal list. `GameSession` updates its private `changeGiven` list and raises `StateChanged`; the view cannot mutate that list.
5. **Inheritance:** when a new customer arrives, the session constructs one of three subclasses. The shared name, gender, and order initialisation runs in the base constructor, so the subclasses contain only the payment line.
6. **Polymorphism again:** the next call to `GeneratePayment` runs the subclass version. The session does not branch on customer type after construction.

Moment in the round	Pillar in action	Mechanism
Button click	Encapsulation	Private session state, public command surface, controlled event invocation.
Passing the piece	Abstraction	Declared as `Denomination`; concrete kind irrelevant to the signature.
Sound and image	Polymorphism	Virtual `PlaySound`; type-driven image path in the registry.
Drawer guard	Encapsulation	`GetPieceCount` reads supply without exposing the list.
New customer	Inheritance	`base(order)` shares initialisation; subclass adds behaviour.
Payment creation	Polymorphism	One abstract call, three concrete strategies.
UI refresh	Abstraction + events	Session publishes data; view renders without knowing the rules.

5.7 Coverage Summary

Pillar	Concrete evidence	Strength
Encapsulation	Private fields and invariants in `Product`, `Order`, `CashDrawer`, and `GameSession`; private `Bill`/`Coin` constructors; event invocation restricted to the publisher; `readonly` generator fields.	Strong, with boundary leaks
Abstraction	Two abstract base classes with one abstract method each; `GameSession` exposes commands and events, not internals; WinForms base controls provide template methods.	Strong conceptually, weak contracts
Inheritance	Three customer subclasses sharing a base constructor; two sealed money subclasses; five UI classes deriving from framework controls; no hierarchy deeper than two.	Clear and purposeful
Polymorphism	Virtual `GeneratePayment` and `PlaySound`; `List<Denomination>`; overridden `OnPaint`/`Dispose`; delegate-based event wiring; one type test and two gender conditionals as counterexamples.	Genuine, with local exceptions

> **SUMMARY FOR A DESIGN REVIEW**
>
> **Encapsulation** is demonstrated by classes that keep their state private and expose behaviour, especially the drawer and the session. **Abstraction** is demonstrated by `Customer` and `Denomination`, which let callers work with "a payer" and "a piece of money" instead of three payer classes and nine money values. **Inheritance** is demonstrated by the shared base constructors and the sealed money leaves. **Polymorphism** is demonstrated every time the session calls `GeneratePayment` or `PlaySound` without knowing the concrete class. The four work together rather than as isolated textbook examples.

CHAPTER 6

6. Design Principles and Code Quality

6.1 SOLID Assessment

SOLID is a set of five design guidelines, not a certification. The program satisfies some of them well and others only partially. The purpose of this section is to make the gaps explicit so that the design can be discussed honestly.

Principle	Verdict	Evidence
Single Responsibility	Mostly met	`Product`, `Order`, `Denomination`, and each generator have one clear job. `GameSession` is the exception: it orchestrates rounds, validates change, owns the score, manages the drawer, and publishes events. Those responsibilities are cohesive around one turn, but six collaborators in one class make it the hardest type to test.
Open/Closed	Partial	Adding a denomination requires only a registry entry; consumers are untouched. Adding a customer behaviour requires a new subclass *and* a change to the factory block in `StartNextCustomer`. The UI layout is also closed to extension: adding a control means editing `GameScreen` and its layout code.
Liskov Substitution	Weak by design	Every customer returns a valid list, but the base contract does not state a sum relationship. `UnderpayingCustomer` intentionally returns less than the total; the caller compensates with an explicit branch. Bill and Coin are safely substitutable for `Denomination`.
Interface Segregation	Not applicable / partial	There are no interfaces to segregate. The event contracts are narrow and well separated. The public method surface of `GameScreen` mixes display methods and event plumbing, but no client is forced to implement anything.
Dependency Inversion	Not met	`GameSession` constructs concrete services itself (`new ProductCatalog()`, `new CashDrawer()`, and so on) and depends on their classes. High-level policy therefore depends on low-level detail. The one inversion-like move is passing a `PaymentGenerator` into `GeneratePayment`, which is dependency injection by parameter.

The most valuable refactor would be to introduce interfaces for the five services and inject them through the `GameSession` constructor. That single change would enable deterministic tests (fake generator, fake drawer) and make the dependency direction explicit, without altering the gameplay.

6.2 Composition and Delegation

Composition is used more often than inheritance, and appropriately so. `GameSession` owns seven collaborators; `Order` owns a list of products; `GameScreen` owns four display controls; `DenominationPanel` owns six bill buttons and three coin buttons. None of these relationships passes the "is-a" test, so inheritance would be wrong.

Responsibility	Owner	Delegated to
Order total	`Order`	Own running sum, updated only by `AddProduct`.
Change arithmetic	`GameSession`	`ChangeChecker` (sum, count, validate).
Piece inventory	`GameSession`	`CashDrawer` (add, remove, count, total).

Responsibility	Owner	Delegated to
Order creation	`GameSession`	`OrderGenerator` + `ProductCatalog`.
Payment composition	`Customer`	`PaymentGenerator`.
Customer names	`Customer`	`NameGenerator`.
Sound playback	`Denomination`	`AudioPlayer` (static cache).
Rendering	`GameScreen`	Child controls, each responsible for its own paint and layout.

Delegation keeps each class small and makes the flow readable: `GiveChange` reads like a recipe ("check customer, check payment, check change, remove pieces, score, next"), while the arithmetic details sit in `ChangeChecker`. The cost is more indirection: to understand one rule, a reader may need to open three files.

6.3 Static Factory and Registry

`Bill` and `Coin` implement a **static registry** (also called a multiton): a private constructor plus a static dictionary of the allowed instances. The type initializer runs on first access, so the nine objects are created lazily and then reused forever. `DenominationPanel` iterates the registry to build its buttons, and `PaymentGenerator` concatenates the two registries into its denomination list.

`AudioPlayer` is a static utility with a private cache. It validates that the caller passed a bare filename rather than a path, combines it with `AppContext.BaseDirectory`, checks existence, then loads and caches one `SoundPlayer` per file under a lock. The filename check is a small but genuine security habit: it prevents a caller from escaping the sound directory with `..\` or an absolute path.

`NameGenerator` is a third static utility: immutable arrays of titles and names, plus one shared `Random`. Static state is convenient here because there is exactly one process and no need to vary the data per session.

Static member	Pattern	Benefit	Cost
`Bill.Denominations`, `Coin.Denominations`	Multiton / registry	Closed legal value set; shared instances; no allocation per piece.	Global mutable dictionary; lazy-initialisation exceptions surface on first use.
`AudioPlayer.PlaySound`	Static cache + lock	Each WAV is loaded once; safe if called from multiple threads.	Hidden global dependency; players are never disposed.
`NameGenerator.GenerateName`	Static utility	No state to pass around; simple call site.	Cannot seed or replace for tests; one shared `Random`.

6.4 Observer Pattern

The relationship between `GameSession` and `GameScreen` is an implementation of the **Observer** pattern using C# events. The session is the subject: it exposes three events and invokes them when its state changes. The form is the observer: it subscribes methods that update the display. `Program.cs` plays the role of the assembler that registers the observer with the subject.

Compared with the classic pattern, the C# version removes the need for an `IObserver` interface and an `Attach` / `Detach` list. A delegate field is the observer list, and `+=` / `-=` are the registration methods. The trade-off is that the contract is a method signature rather than a named interface, so the compiler cannot tell you which

methods are valid observers beyond matching parameter types.

Role	Type	Details
Subject	`GameSession`	Raises `StateChanged`, `MessageSent`, `CustomerStarted`.
Observer	`GameScreen`	Subscribes `ShowState` and `ShowMessage`; a lambda handles `CustomerStarted`.
Assembler	`Program`	Wires both directions before starting the session.
Notification payload	Method arguments	No event args class; values are passed positionally.

Benefits. The session has no reference to the UI, so the rules can be tested or reused with another view. The form has no reference to the session, so controls can be rearranged without touching game logic. New observers (a log window, a sound controller) could be added in `Program` without changing either class.

Costs. Control flow is not visible at the call site: a method that calls `SendState()` indirectly runs UI code. Exceptions in a subscriber propagate back into the publisher. There is no unsubscribe, so a short-lived view would leak. And the six-argument `StateChanged` signature is easy to misuse; a dedicated `GameState` type would make the notification self-describing.

6.5 Resource Management

WinForms and GDI+ hold unmanaged resources, so image lifetime matters. `CustomerDisplay` handles it carefully: it loads the file inside a `using` block, clones it into a `Bitmap`, disposes the previously displayed image, and overrides `Dispose(bool)` to release the last one. The clone is deliberate: `Image.FromFile` keeps the file locked until the image is disposed, while a clone is independent of the file.

UI/CUSTOMERDISPLAY.CS — BOUNDED IMAGE LIFETIME

```
using Image sourceImage = Image.FromFile(imagePath);
Image nextImage = new Bitmap(sourceImage);
Image? previousImage = customerImage;
customerImage = nextImage;
previousImage?.Dispose();
Invalidate();

// ...
protected override void Dispose(bool disposing)
{
    if (disposing)
        customerImage?.Dispose();
    base.Dispose(disposing);
}
```

Other resources are less strict. `DenominationButton` loads an image and never disposes it; there are only nine buttons, so the leak is bounded and lasts for the process lifetime. `OrderDisplay` loads a receipt background and does not dispose it. Each `DenominationButton` creates a `ToolTip`, which is a component and should be disposed but is not. `AudioPlayer` caches `SoundPlayer` instances without disposing them; again, the cache is process-wide and small. These are acceptable for a desktop toy but would be flagged in a long-running application.

One positive detail is that `AudioPlayer` calls `player.Load()` when caching, so the first play does not pay the disk-read cost. `SoundPlayer.Play()` is asynchronous and returns immediately, which keeps the UI responsive.

6.6 Coupling and Cohesion

Cohesion is high in the small types: `Product` only holds product data, `ChangeChecker` only does arithmetic, and `NameGenerator` only composes names. `GameSession` has lower cohesion because it is the orchestrator, but its methods all serve the same round. `GameScreen` has the lowest cohesion: it builds five control trees, computes an absolute layout, re-raises four events, and formats three kinds of display. At 272 lines it is still readable, but it is the class most likely to grow into a maintenance problem.

Relationship	Coupling	Comment
`Program` → everything	High (expected)	The composition root must know both halves; that is its job.
`GameScreen` → `GameSession`	Very low	No direct reference; only matching event signatures.
`GameSession` → UI	None	The session does not import WinForms.
`GameSession` → services	High	Constructs and calls five concrete services.
Services → models	Medium	Expected; services operate on domain types.
Models → services	Medium	Two upward edges: `PaymentGenerator` and `AudioPlayer`.
UI controls → models/services	Medium	`DenominationPanel` reads money registries; `GameScreen` calls `AudioPlayer` to play the button sound.

At the namespace level, `Models.Customers` and `Models.Money` both depend on `Services`, while `Services` depends on `Models`. The cycle is small and harmless in a single project, but it is the main structural deviation from the layered diagram in Chapter 2.

6.7 Observations and Refactoring Roadmap

The following findings are ordered by the value they would add, not by how easy they are to fix. None of them prevents the program from working; they are the difference between "a project that runs" and "a project that can be maintained and tested".

ID	Observation	Impact	Priority
R1	No interfaces and no constructor injection; `GameSession` news up all services.	Cannot substitute fakes; unit tests would require a running UI or reflection.	High
R2	`StateChanged` passes six positional values, including two adjacent integers.	Easy to swap `amountPaid` and `changeGiven`; every change touches all subscribers.	High

ID	Observation	Impact	Priority
R3	Image paths are relative to the current working directory; `AudioPlayer` uses `AppContext.BaseDirectory`.	Launching from the repository root throws `FileNotFoundException` before the window opens. Confirmed during this review.	High
R4	`ApplicationConfiguration.Initialize()` is commented out in `Program.cs`.	The visual-styles and high-DPI settings declared in the project file are never applied; rendering may be blurry on scaled displays.	High
R5	No automated tests.	Rules are verified manually; regressions after refactoring are likely.	High
R6	Mutable getters and setters: `Order.GetProducts`, `Bill.Denominations`, `Denomination.Value`, `ImagePath`.	Callers can break invariants or mutate shared money objects.	Medium
R7	Customer type probabilities and factory logic are hard-coded in `StartNextCustomer`.	Adding or tuning a customer type requires editing the session; difficulty is not configurable.	Medium
R8	Drawer can run out of small coins with no exchange action.	Possible unwinnable round even though the drawer holds enough total value.	Medium
R9	No persistence: score and settings vanish on exit.	Short sessions; no high-score incentive.	Medium
R10	Magic numbers for layout, probabilities, and starting counts.	Harder to tune; values are duplicated or unexplained.	Low
R11	`if (denomination is Bill)` in `DenominationButton`.	Type test in place of polymorphism; adding a new money kind requires editing the button.	Low
R12	`Gender` is in the global namespace; one file omits its namespace entirely.	Namespace inconsistency; domain detail leaks globally.	Low
R13	Three separate unseeded `Random` instances (customer, session, generators).	Non-reproducible tests and debugging; not thread-safe if ever shared.	Low
R14	Some controls and images are never disposed.	Bounded leak for a short-lived desktop app; not a practical problem today.	Low

Suggested sequence

Phase	Goal	Work items
1 — Hygiene	Remove easy failure modes.	Restore `ApplicationConfiguration.Initialize()`; resolve assets from `AppContext.BaseDirectory`; make money properties immutable; return read-only collections; fix the namespace inconsistency.

Phase	Goal	Work items
2 — Testability	Make rules verifiable without the UI.	Extract `IChangeChecker`, `ICashDrawer`, `IPaymentGenerator`, `IOrderGenerator`; inject them into `GameSession`; replace the six-argument event with a `GameState` record; inject a seeded `Random`.
3 — Gameplay	Increase depth and fairness.	Add bill breaking or a restock rule; make the customer mix configurable; add a timer or move-limit mode; persist a high score; add a sound toggle.
4 — Polish	Broaden accessibility and reach.	Keyboard activation for denomination buttons; a single-click mode; screen-reader announcements for score and total; scalable layout; localization for Filipino and English.

CHAPTER 7

7. User Interface

7.1 Screen Composition

The main window has a fixed logical size of 1280×850 pixels and a minimum of 1000×700. It uses an *overlay* layout rather than docked panels: the customer portrait occupies the left 54% of the client area and runs the full height; the header, receipt, and action buttons occupy the right side; the cash drawer spans the full width across the bottom and sits on top of the portrait area. The arrangement is intended to resemble a counter: customer on the left, receipt and till controls on the right, drawer in front of the cashier.

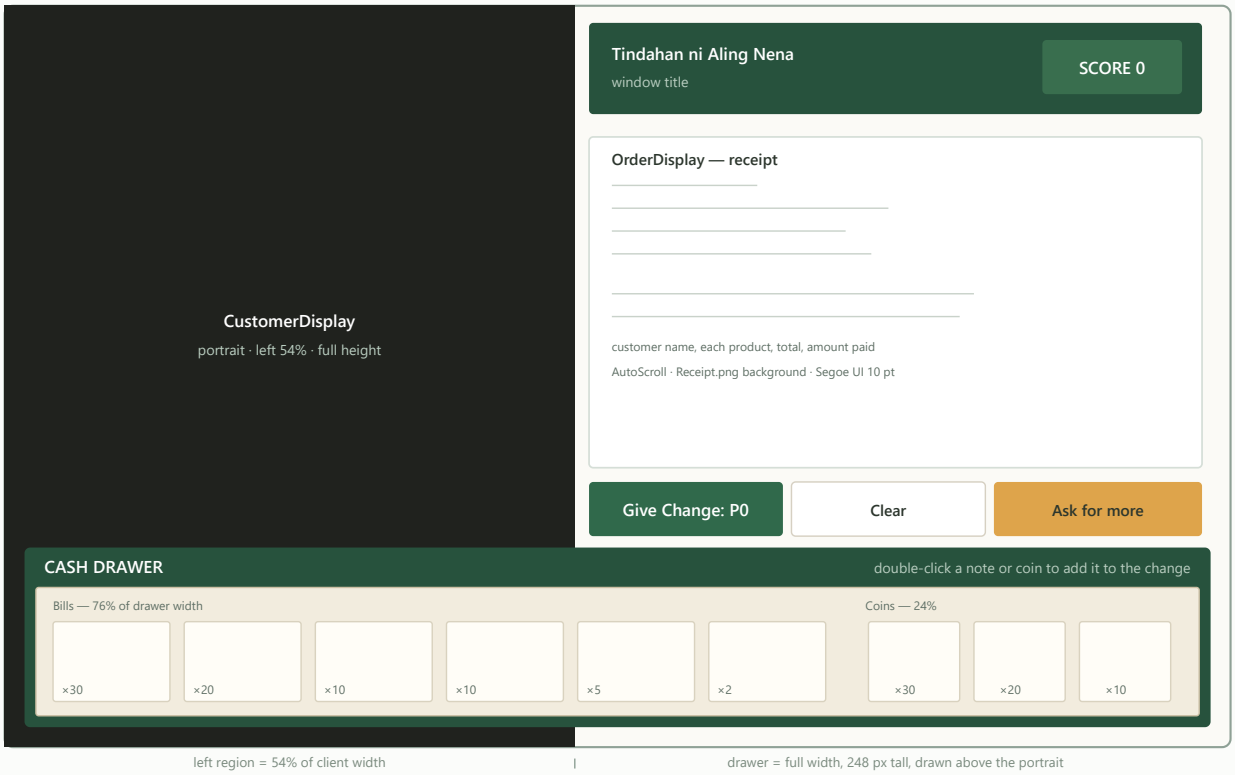


Figure 4. Schematic screen composition (not to scale). The cash drawer overlaps the lower part of the customer portrait; the header and order panel are positioned in the right region. The actual control tree is described in the control table below.

7.2 Layout and Resizing

`GameScreen` adds four top-level children: `customerDisplay`, `header`, `orderPanel`, and `denominationPanel`. The order panel itself contains the receipt and the three-button row. Positions are recomputed in `LayoutOverlayControls`, which is called once from the constructor and again on every `Resize` event.

Element	Bounds rule	Notes
Customer portrait	`(0, 0, width × 0.54, height)`	Sent to the back so the drawer can overlap it.
Header	`(customerWidth + 14, 12, remainingWidth, 76)`	Contains the title and the score card.

Element	Bounds rule	Notes
Cash drawer	`(14, height - 248 - 14, width - 28, 248)`	Full-width strip; the dominant visual anchor.
Order panel	`(customerWidth + 14, 100, remainingWidth, drawerTop - 114)`	Height shrinks as the window shrinks, with a floor of 200 px.
Buttons	Three equal columns inside the order panel	68 px row reserved at the bottom of the panel.

The approach is direct and predictable, but it has two consequences. First, layout is not data-driven: to add a control, a developer edits the constructor and the layout method. Second, the layout runs without `SuspendLayout` / `ResumeLayout`, so a live resize can visibly flicker. WinForms' docking and table layouts are used inside the panels and drawer, so most of the fine-grained arrangement is still handled by the framework.

7.3 Controls and Responsibilities

Control	Base class	Responsibility	Public surface
`GameScreen`	`Form`	Window shell, overlay layout, event mediation, display API.	Four events; `ShowState`, `ShowMessage`, `ShowCustomer`, `ClearCustomer`.
`CustomerDisplay`	`PictureBox`	Loads a portrait and paints it with cover scaling; disposes images.	`DisplayCustomer`, `DisplayBackground`.
`OrderDisplay`	`Panel`	Renders the receipt text over a background image; scrolls when tall.	`ShowOrder(order, name, paid)`.
`DenominationPanel`	`TableLayoutPanel`	Builds bill and coin slots; shows counts; raises clicks.	`ShowPieceCounts`, `DenominationClicked`.
`DenominationButton`	`PictureBox`	Displays one money image; rotates bills; tooltip and accessible name.	`GetDenomination`.

`DenominationButton` is a `PictureBox` rather than a `Button`, which gives full control over the image but also means the control does not get standard button keyboard behavior. Activation is bound to `DoubleClick`, and the drawer's instruction text explicitly tells the player to double-click. This is discoverable but unusual; most users expect a single click or a space/enter activation on a focused control.

7.4 Event Wiring

Eight subscriptions in `Program.cs` connect the two halves. The chain for a denomination click illustrates the path:

1. `DenominationButton.DoubleClick` fires inside a slot.
2. `DenominationPanel.OnDenominationButtonClicked` casts the sender and raises `DenominationClicked(denomination)`.
3. `GameScreen.OnPanelDenominationClicked` re-raises its own `DenominationClicked`.

4. `GameSession.AddChangePiece` receives the piece because `Program` subscribed it.
5. The session updates its state and raises `StateChanged`.
6. `GameScreen.ShowState` updates the receipt, button caption, score, and drawer counts.

The action buttons follow a shorter path: the click handler first plays `button.wav` (an extra subscription added inside `AddActionButton`), then invokes the UI event. This guarantees consistent click feedback even when the resulting action is a failure. The `Give Change` button's caption is also the live change total, so the player can see the sum before committing.

7.5 Visual System

Token	Value	Used for
Canvas	`#F6F2E8`	Form background and button strip.
Ink	`#2C362D`	Primary text and borders.
Deep green	`#27523D`	Header, cash drawer, primary surfaces.
Counter green	`#30694B`	Give Change button, accents.
Score green	`#396E4E`	Score card.
Receipt cream	`#F2ECDE`	Drawer interior.
Slot white	`#FFFDF7`	Individual denomination compartments.
Gold	`#DEA44B`	Ask for more button, section rules.
Customer stage	`#20221E`	Portrait background.

Typography is Segoe UI throughout: 20 pt bold for the title, 12 pt bold for the score, 10 pt bold for buttons and headings, 10 pt regular for the receipt, and 8–9 pt for counts and hints. The palette is intentionally low-saturation, with gold reserved for the secondary action and green for confirmation. The result is legible and consistent, though it is entirely hard-coded in C# rather than defined as a shared theme resource.

7.6 Accessibility

The program makes a genuine start on accessibility. Each denomination button sets `AccessibleName` ("₱20") and `AccessibleDescription` ("Add a 20 peso denomination to the change"), and each count label updates its `AccessibleName` to a phrase such as "3 pieces of 20 pesos". Tooltips repeat the value. The receipt uses a real `Label`, so its text is available to screen readers, and the score is a label rather than a painted graphic.

The gaps are more significant than the wins. `PictureBox` is not a focusable button, so a keyboard-only user cannot tab to a denomination or activate it; there is no `KeyDown` handler, no access key, and no focus rectangle. Double-click activation is a motor-accessibility barrier. The fixed font sizes do not scale with system text settings, and the layout assumes a minimum width of 1000 pixels. The drawer counts are visually tiny. A future pass should add keyboard focus and `Enter` / `Space` activation, support single-click as an option, announce score changes through an accessible live region, and offer a high-contrast theme.

7.7 Assets and Rendering

The project file includes every file under `Assets/` and copies it to the output directory with `PreserveNewest`. At runtime, `AudioPlayer` resolves sounds from `AppContext.BaseDirectory`, but the image code uses paths relative to the process working directory: `Path.Combine("Assets", ...)`. The inconsistency is the cause of the confirmed launch failure described in R3. Running the executable from its output folder works; running it from the repository root does not.

Asset	Consumer	Rendering detail
Bill images (6 JPG)	`DenominationButton`	Rotated 90 degrees so the note stands upright; width and height swapped after rotation.
Coin images (3 PNG)	`DenominationButton`	Displayed at 16% scale, centred in a slot, transparent background.
Customer portraits (10 JPG)	`CustomerDisplay`	Scaled to cover the control, centred, cropped; drawn with bicubic interpolation; double-buffered by WinForms.
`Receipt.png`	`OrderDisplay`	Stretched as the panel background; text is a transparent label on top.
WAV files (5)	`AudioPlayer`	Loaded once, cached per filename, played asynchronously.

The customer portraits are indexed 1–10, with men at 1, 3, 5, 6, 9, 10 and women at 2, 4, 7, 8. Only the `Normal` state folder exists; `CustomerState.Happy` and `Angry` are declared but never used, and no assets exist for them. `DisplayCustomer` returns the chosen number, but the caller in `Program` discards it, so the same customer number is not retained for a later mood change.

CHAPTER 8

8. User Guide

Objective

Serve each customer correctly and keep the streak alive. A customer orders one or more items; the receipt shows every product, the total, and the amount the customer has handed over. Your job is to take notes and coins from the cash drawer until the selected change exactly equals the overpayment, then press **Give Change**. A correct answer increases the score and brings the next customer. A wrong answer clears your selection and resets the score to zero.

Controls

Action	How	Effect
Add a note or coin	Double-click its picture in the cash drawer.	Appends one piece to the selected change and plays a paper or coin sound. The count on the slot is the number still available.
Give change	Click the green **Give Change: P<sum>** button.	Validates the selected sum and either advances to the next customer or reports an error.
Clear selection	Click **Clear**.	Returns every selected piece to the drawer conceptually; the list is emptied without penalty.
Ask for more money	Click the gold **Ask for more** button.	Only useful when the customer has paid less than the total. Adds another generated payment and puts it in the drawer.

Playing a round

1. Read the customer's name and order on the receipt. Confirm the total.
2. Read the line *Customer paid*. If it is lower than the total, press **Ask for more** first.
3. Subtract the total from the amount paid. That difference is the change you owe.
4. Double-click notes and coins whose values add up to that difference. Watch the **Give Change** caption; it shows the running sum.
5. If you overshoot or change your mind, press **Clear** and start again. Clearing is free.
6. Press **Give Change**. Success increments the score; failure resets it to zero but keeps the same customer.

> **WORKED EXAMPLE**
>
> Order: Pandesal ₱2 + Instant Noodles ₱12 + Candy ₱1 = **₱15**. The customer pays **₱50**. Change due = ₱35. Suitable combinations include ₱20 + ₱10 + ₱5, or ₱10 + ₱10 + ₱10 + ₱5, or ₱20 + ₱5 + ₱5 + ₱5. Any combination summing to ₱35 is accepted, provided the drawer holds the pieces. The button reads **Give Change: P35** before you confirm.

Scoring and strategy

- **Score +1** for every correct change. A wrong answer resets the score to zero; there is no partial credit.

- **Order size grows with the score.** At score 0 the customer buys 1–4 items; at score 10, 1–14. Expect longer receipts and larger totals as you progress.
- **Conserve small coins.** The drawer can run short of ₱1, ₱5, and ₱10 coins if you always spend them. Prefer larger pieces when the arithmetic allows, so you keep the flexibility to make odd amounts later.
- **Use Clear liberally.** Rebuilding a selection costs nothing; a wrong confirmation costs the streak.
- **Check underpayments first.** If the paid amount is below the total, no change combination can succeed until you ask for more.
- **There is no timer.** Take the time to add carefully; the game rewards accuracy, not speed.

Common mistakes

Symptom	Cause	Fix
"The customer didn't pay enough! Ask for more money."	You pressed Give Change while the payment was below the total.	Press Ask for more, then recalculate the change.
"The customer already paid enough!"	You pressed Ask for more when the payment already covered the total.	Give the change instead.
"No more P20 left in the drawer!"	You selected more pieces of that value than the drawer holds, counting what you have already selected.	Clear some selections or use a different combination.
"Wrong change! Try again."	The selected sum did not equal paid minus total.	Clear, recalculate, and rebuild the change. The score has already reset.

CHAPTER 9

9. Build and Run

9.1 Prerequisites

- Windows 10 or Windows 11 (the project targets `net8.0-windows` and uses WinForms and GDI+).
- The .NET 8 SDK, or a newer SDK that can target .NET 8, together with the Windows Desktop targeting pack. The Microsoft-suggested installation is the ".NET desktop development" workload in Visual Studio 2022, or the standalone .NET SDK.
- No NuGet packages are required; restore only resolves the framework reference.
- If you build the `.slnx` solution directly, use an SDK that understands the XML solution format (SDK 9.0.200 or later; Visual Studio 2022 17.13 or later). Older tooling can open the `.csproj` directly.

The review environment used SDK 10.0.402. Both the project and the solution built successfully with zero warnings and zero errors.

9.2 Commands

POWERSHELL — BUILD FROM THE REPOSITORY ROOT

```
# restore and build the solution
dotnet restore .\TindahanNiAlingNena_Recoded.slnx
dotnet build   .\TindahanNiAlingNena_Recoded.slnx -c Release

# or build the project directly if your SDK does not support .slnx
dotnet build .\TindahanNiAlingNena_Recoded\TindahanNiAlingNena_Recoded.csproj -c Release
```

Running requires the correct working directory. The image loaders use paths such as `Assets/Bills/PHP_20.jpg` relative to the process working directory, not relative to the executable. Running the project from the repository root with `dotnet run --project ...` therefore throws `FileNotFoundException` during form construction (confirmed). Use one of the following forms instead:

POWERSHELL — THREE WAYS THAT WORK

```
# 1. run from inside the project folder (source Assets are present)
Push-Location .\TindahanNiAlingNena_Recoded
dotnet run --no-build
Pop-Location

# 2. run the built executable from its output folder (Assets copied there)
.\TindahanNiAlingNena_Recoded\bin\Debug\net8.0-windows\TindahanNiAlingNena_Recoded.exe

# 3. launch through Visual Studio with the project as the working directory
#    (the default for a WinForms project)
```

In Visual Studio, press **F5** or **Ctrl+F5**. The project's default working directory is the project folder, which contains the source `Assets` tree, so the relative paths resolve.

9.3 Project Configuration

Property	Value	Effect
`OutputType`	`WinExe`	Builds a GUI executable without a console window.
`TargetFramework`	`net8.0-windows`	Requires a Windows desktop runtime.
`UseWindowsForms`	`true`	Adds the WinForms framework reference and designer support.
`ImplicitUsings`	`enable`	Removes repetitive `using System;` declarations.
`Nullable`	`enable`	Enables nullable reference analysis; the build is warning-free.
`ApplicationVisualStyles`	`true`	Generates a setting for `ApplicationConfiguration`, but the call is commented out in `Program.cs`.
`ApplicationHighDpiMode`	`SystemAware`	Same caveat: not applied because initialisation is skipped.
`ApplicationDefaultFont`	Segoe UI, 9 pt	Only applied through the generated initialiser.
`Content Include="Assets\**\*"`	`CopyToOutputDirectory=PreserveNewest`	Copies images and sounds beside the executable.

> **KNOWN LAUNCH TRAP**
>
> Because `ApplicationConfiguration.Initialize()` is commented out, the visual-style, DPI, and default-font settings in the project file have no effect at runtime. Restoring the call is a one-line fix that should be made before evaluating high-DPI behaviour.

9.4 Publishing

For a redistributable build, publish the project for a specific Windows architecture. A framework-dependent build is small but requires the .NET Desktop Runtime on the target machine; a self-contained build includes the runtime.

POWERSHELL — PUBLISH OPTIONS

```
# framework-dependent, 64-bit Windows
dotnet publish .\TindahanNiAlingNena_Recoded\TindahanNiAlingNena_Recoded.csproj `
  -c Release -r win-x64 --self-contained false -o .\publish

# self-contained single-folder build
dotnet publish .\TindahanNiAlingNena_Recoded\TindahanNiAlingNena_Recoded.csproj `
  -c Release -r win-x64 --self-contained true -o .\publish-selfcontained
```

Run the published executable from inside the publish folder so that the copied `Assets` tree is found. The project does not currently produce a single-file bundle; adding `PublishSingleFile` would also require care with the image paths.

CHAPTER 10

10. Testing and Quality Assurance

10.1 Current Test Posture

The repository contains no test project, no test framework reference, and no automated assertions. Quality assurance to date has been manual play-testing plus a successful compilation. The review build produced zero warnings and zero errors, which confirms type correctness and nullable discipline but says nothing about the game rules. Because most rules are deterministic given their inputs, the code is unusually easy to test — the absence of tests is an opportunity, not a limitation imposed by the design.

10.2 Manual Test Checklist

ID	Scenario	Expected result
T1	Launch the executable from its output folder.	Window opens at 1280×850, a customer portrait appears, the drawer shows the starting counts, score is 0.
T2	Launch from the repository root.	`FileNotFoundException` before the window appears (known defect R3).
T3	Exact-paying customer; press Give Change with no selection.	Success sound; score becomes 1; next customer appears.
T4	Overpaying customer; select the exact overpayment.	Success; selected pieces disappear from the drawer counts; score increments.
T5	Overpaying customer; select the wrong sum.	Failure sound and message; selection cleared; score resets to 0; same customer remains.
T6	Underpaying customer; press Give Change immediately.	Refusal: "The customer didn't pay enough! Ask for more money."; score unchanged.
T7	Underpaying customer; press Ask for more.	Payment increases above the total; drawer counts increase; change can now be given.
T8	Paid customer; press Ask for more.	Refusal: "The customer already paid enough!".
T9	Double-click a denomination until the count reaches zero, then once more.	Further selections are refused with "No more P<value> left in the drawer!".
T10	Select pieces, then press Clear.	Selection empties; drawer counts restore; score unchanged.
T11	Resize the window down to the minimum size.	All regions remain visible; receipt panel keeps at least 200 px of height; no overlap with the drawer.
T12	Build a score of 5 or more and inspect a new order.	Orders can exceed four items; receipt scrolls; totals match the listed products.
T13	Click each action button.	A button click sound plays before the action; captions and colors remain readable.
T14	Delete or rename a sound file, then trigger it.	Unhandled `FileNotFoundException`; the app does not degrade gracefully (known limitation).

10.3 Suggested Unit Tests

A test project should target the rules, not the form. The following cases are directly expressible against the current public APIs and would catch the most likely regressions.

Target	Test	Assertion
`PaymentGenerator`	Exact payment for 1, 15, 100, 1000.	Sum of returned values equals the target.
	Underpayment for 100, repeated many times.	Sum is in `[0, 99]`; for amount 1 the list is empty.
	Overpayment for 100, repeated many times.	Sum is greater than 100 and at most 150.
	Non-positive input.	Throws `ArgumentOutOfRangeException`.
`ChangeChecker`	Total of mixed bills and coins.	Correct integer sum.
	Change exactly equal to overpayment.	`IsChangeCorrect` returns true.
	One peso short or over.	Returns false.
`CashDrawer`	Starting total and counts.	₱880; 30 ones, 20 fives, 10 tens, 10 twenties, 5 fifties, 2 hundreds.
	Add and remove a piece.	Count changes by one; removing an absent value returns false.
	Remove from empty drawer.	Returns false and leaves the drawer unchanged.
`Order`	Add several products.	Total equals the sum of prices; count matches.
	Empty order.	Total 0, count 0.
`Customer`	Each subclass against a fixed order.	Payment sum is equal, greater, or less than the total as specified.
	Name and gender are populated.	Name is non-empty; gender is one of the two enum values.
	Generated name matches the gender's title list.	Starts with a title from the correct array.
`GameSession`	With injected fakes, play a full winning round.	Score increments once; drawer counts reflect payment and change; next customer requested.

Two structural changes would make these tests much easier. First, inject the services and a seeded `Random` so that a test can predict the customer type and payment. Second, expose a read-only snapshot of the session state (the proposed `GameState` record) so assertions do not need to reach into private fields or simulate the UI.

For the payment builder, a useful property-based test is: *for every target from 1 to 2,000, the returned list terminates and sums exactly to the target*. The guarantee follows from the ₱1 coin, but the test documents it and would catch a future change to the fitting rule.

CHAPTER 11

11. Limitations and Future Work

11.1 Known Limitations

Area	Limitation	Consequence
Launch	Image assets are resolved against the working directory, unlike sounds which use `AppContext.BaseDirectory`.	Running from the wrong folder crashes before the window opens (confirmed).
Initialisation	`ApplicationConfiguration.Initialize()` is commented out.	Visual styles, DPI mode, and default font settings are not applied.
Gameplay	No loss condition, timer, or move limit.	The game is an endless accuracy drill rather than a challenge with stakes.
Gameplay	The drawer can run out of small coins while holding enough total value.	A round can become impossible; there is no bill-breaking action or restock rule.
Gameplay	Wrong change resets the entire score.	One mistake erases a long streak; no partial credit.
Progression	Only order size scales with score; customer mix and probabilities are fixed.	Later rounds are longer, not mechanically different.
Persistence	No save file, settings, or high-score table.	Every session starts from zero and nothing survives exit.
Content	Six products, nine denominations, ten portraits, three customer behaviours.	Repetition becomes noticeable quickly; names and faces recur.
Content	Only the `Normal` customer state has artwork; `Happy` and `Angry` are unused.	The feedback loop is text and sound only; the portrait never reacts.
Accessibility	Denomination buttons are not focusable; activation requires a double-click.	Keyboard-only and motor-impaired users cannot play.
Accessibility	Fixed font sizes and a 1000 px minimum width.	Poor behavior under text scaling and on small displays.
Robustness	Missing assets throw unhandled exceptions.	A partial install or corrupt file terminates the app.
Architecture	No interfaces; services are constructed inside `GameSession`.	Hard to unit test or replace.
Architecture	Six positional arguments in `StateChanged`.	Fragile contract; easy to transpose the two integer amounts.
Architecture	Mutable internals escape through getters and setters.	Invariants can be broken by future callers.

Area	Limitation	Consequence
Quality	No tests, no CI, no logging.	Regressions are discovered by playing; production issues leave no trace.
Localization	UI text and product names are fixed; no resource files.	Cannot switch between Filipino and English.

11.2 Improvement Roadmap

The following sequence balances quick wins against deeper structural work. The first two items remove failure modes that affect every run; the rest increase depth and maintainability.

Step	Change	Benefit
1	Resolve every asset from `AppContext.BaseDirectory`; restore `ApplicationConfiguration.Initialize()`.	Reliable launch from any folder and correct DPI/visual styles.
2	Make `Denomination` properties immutable and return read-only collections from `Order` and the registries.	Protects the shared money objects and the order invariant.
3	Introduce service interfaces and constructor injection; inject a seeded `Random`.	Deterministic unit tests and a clear dependency direction.
4	Replace the six-argument event with an immutable `GameState` record.	Self-documenting notifications and safer evolution.
5	Add a bill-breaking action or a restock rule when small denominations run out.	Removes the unwinnable-drawer edge case.
6	Add a mode with a timer or a limited number of mistakes, and persist a high score.	Creates stakes and a reason to replay.
7	Support keyboard activation, single-click mode, scalable fonts, and a high-contrast theme.	Makes the game usable by a wider audience.
8	Add mood artwork for happy and angry states and animate the portrait on success or failure.	Strengthens feedback and uses the existing `CustomerState` enum.

> **WHAT TO PRESERVE**
>
> Any refactor should keep the two properties that make the current design teachable: the domain model stays free of WinForms types, and the view stays passive. Adding interfaces and tests should not turn the program into a framework. The clarity of the money hierarchy and the event-driven split is the project's main strength.

APPENDIX A

Appendix A: Class Reference

Public surface of every type, in dependency order. "Abstract" and "sealed" refer to the class modifiers; method signatures omit access modifiers unless they are not public.

Type	Modifiers	Public members and role
`Customer`	abstract	`GetName()`, `GetGender()`, `GetOrder()`, abstract `GeneratePayment(PaymentGenerator)`. Owns order, gender, name.
`ExactPayingCustomer`	class	Constructor `(Order)`; overrides `GeneratePayment` to return an exact sum.
`OverpayingCustomer`	class	Constructor `(Order)`; overrides `GeneratePayment` to return more than the total.
`UnderpayingCustomer`	class	Constructor `(Order)`; overrides `GeneratePayment` to return less than the total.
`Denomination`	abstract	`Value`, `ImagePath`, abstract `PlaySound()`.
`Bill`	sealed	Private constructor; `PlaySound()`; static `Denominations: Dictionary<int,Bill>` with 20, 50, 100, 200, 500, 1000.
`Coin`	sealed	Private constructor; `PlaySound()`; static `Denominations: Dictionary<int,Coin>` with 1, 5, 10.
`Product`	class	Constructor `(string name, int price)`; `GetName()`, `GetPrice()`.
`Order`	class	`AddProduct(Product)`, `GetProducts()`, `GetProductCount()`, `GetTotalCost()`.
`ProductCatalog`	class	`GetRandomProduct()`, `GetRandomProducts(int count)`.
`OrderGenerator`	class	Constructor `(ProductCatalog)`; `GenerateOrder(int min, int max)`.
`PaymentGenerator`	class	`GenerateExactPayment`, `GenerateUnderPayment`, `GenerateOverPayment`; private `BuildPayment`.
`ChangeChecker`	class	`GetTotal(List<Denomination>)`, `CountPieces(list, value)`, `IsChangeCorrect(total, paid, change)`.
`CashDrawer`	class	`AddPiece`, `AddPieces`, `RemovePiece(int value)`, `GetPieceCount`, `GetPieceCounts`, `GetTotalAmount`.
`NameGenerator`	static	`GenerateName(Gender)`; title and name pools.
`AudioPlayer`	static	`PlaySound(string fileName)`; private cached `SoundPlayer` dictionary.
`GameSession`	class	Events `StateChanged`, `MessageSent`, `CustomerStarted`; `Start`, `AddChangePiece`, `ClearChange`, `GiveChange`, `AskForMore`; private round helpers.

Type	Modifiers	Public members and role
`GameScreen`	class : Form	Events `DenominationClicked`, `GiveChangeClicked`, `ClearClicked`, `AskForMoreClicked`; `ShowState`, `ShowMessage`, `ShowCustomer`, `ClearCustomer`.
`CustomerDisplay`	class : PictureBox	`DisplayCustomer(CustomerState, int?, Gender?)`, `DisplayBackground()`; overrides `OnPaint` and `Dispose`.
`OrderDisplay`	class : Panel	`ShowOrder(Order, string customerName, int amountPaid)`.
`DenominationPanel`	class : TableLayoutPanel	Event `DenominationClicked`; `ShowPieceCounts(IReadOnlyDictionary<int,int>)`.
`DenominationButton`	class : PictureBox	Constructor `(Denomination)`; `GetDenomination()`.
`Program`	top-level	Wires events, starts the session, runs the form. No public type beyond the generated entry point.

APPENDIX B

Appendix B: File Map

All source files with physical line counts (including blanks and comments). The totals below are the basis for the 1,456-line figure used throughout this document.

File	Lines	Purpose
`Program.cs`	21	Entry point, event wiring, session start.
`Models/Product.cs`	24	Name and price value object.
`Models/Order.cs`	29	Product list and running total.
`Models/Customers/Customer.cs`	51	Abstract customer, `Gender` enum, name/gender initialisation.
`Models/Customers/ExactPayingCustomer.cs`	17	Exact payment strategy.
`Models/Customers/OverpayingCustomer.cs`	17	Overpayment strategy.
`Models/Customers/UnderpayingCustomer.cs`	17	Underpayment strategy.
`Models/Money/Denomination.cs`	10	Abstract money base.
`Models/Money/Bill.cs`	30	Sealed bill type and six-value registry.
`Models/Money/Coin.cs`	28	Sealed coin type and three-value registry.
`Services/GameSession.cs`	179	Round state, rules, scoring, events.
`Services/ProductCatalog.cs`	36	Fixed six-product list and random selection.
`Services/OrderGenerator.cs`	28	Random order builder.
`Services/PaymentGenerator.cs`	85	Denomination composition and three payment methods.
`Services/ChangeChecker.cs`	42	Sum, count, and exact-change validation.
`Services/CashDrawer.cs`	97	Piece inventory with starting stock of ₱880.
`Services/NameGenerator.cs`	42	Filipino title and first-name pools.
`Services/AudioPlayer.cs`	37	Validated, cached WAV playback.
`UI/GameScreen.cs`	272	Window shell, overlay layout, event mediation.
`UI/CustomerDisplay.cs`	109	Portrait loading, cover painting, disposal.
`UI/OrderDisplay.cs`	61	Receipt formatting and background.
`UI/DenominationPanel.cs`	184	Drawer grid, slots, counts, click event.
`UI/DenominationButton.cs`	40	Money image, rotation, tooltip, accessibility.
Total	**1,456**	**23 files**

Counts are physical lines as reported by the development environment; they include using directives, blank lines, and closing braces.

APPENDIX C

Appendix C: Glossary

Abstraction. A simplified model that exposes what is essential and hides implementation detail. In this project, `Customer` and `Denomination`.

Aggregation. A whole–part relationship in which the parts can exist independently. `CashDrawer` aggregates shared denomination objects.

Association. A structural link between objects. `Customer` is associated with one `Order`.

Cohesion. How strongly the responsibilities inside one class belong together. High cohesion is desirable.

Composition. A whole–part relationship in which the whole owns its parts. `Order` composes its product list; `GameSession` composes its services.

Constructor. The special member that initialises a new object. `Bill` and `Coin` use private constructors to control creation.

Coupling. The degree to which one type depends on another. Low coupling between UI and session is achieved with events.

Delegate. A type-safe reference to a method. `Action<T>` is used for every event in the project.

Dependency inversion. Depending on abstractions rather than concrete implementations. Not applied in this project.

Encapsulation. Bundling state with the methods that maintain it and restricting direct access. See private fields and controlled events.

Event. A member that lets a publisher notify subscribers. `StateChanged` is the central example.

Flyweight / multiton. Sharing a small set of instances instead of creating many. The nine denomination objects behave this way.

Inheritance. Deriving a subtype from a base type. `OverpayingCustomer` inherits from `Customer`.

Instance. A concrete object created from a class. The drawer holds many references but only nine distinct denomination instances.

Interface. A contract without implementation. The project defines none, which limits substitution and testing.

Liskov substitution. Subtypes should be usable wherever the base type is expected without breaking the caller's assumptions.

Multiton. A pattern that restricts a class to a fixed, named set of instances, one per key. See `Bill.Denominations`.

Nullable reference types. A C# feature, enabled here, that distinguishes nullable from non-nullable references at compile time.

Object. A runtime instance of a class, with state and behaviour.

Observer. A pattern in which a subject notifies registered observers of changes. Implemented with C# events.

Override. Replacing a virtual or abstract base implementation. `GeneratePayment` is overridden three times.

Polymorphism. One call site, many runtime behaviours. `piece.PlaySound()` is the clearest example.

Property. A member that exposes a field through get/set accessors. `Value` and `ImagePath` are properties.

Sealed. A modifier that prevents further inheritance. Applied to `Bill` and `Coin`.

SOLID. Five design principles: single responsibility, open/closed, Liskov substitution, interface segregation, dependency inversion.

Static. A member that belongs to the type rather than an instance. Registries, generators, and the audio cache are static.

UML. Unified Modeling Language, used here for the class diagram and relationship vocabulary.

Virtual dispatch. The runtime selection of an overridden method based on the object's actual type.

APPENDIX D

Appendix D: References

Source and project

- Project repository: `https://github.com/kvkbkbb/TindahanNiAlingNena_Recoded`
- Source baseline: commit `8795221` ("added stuff agaaaain patapos naaaaa").
- Local build verified with .NET SDK 10.0.402 targeting `net8.0-windows`.

Language and platform documentation

- Microsoft Learn, *Object-Oriented Programming (C#)*: `https://learn.microsoft.com/dotnet/csharp/fundamentals/object-oriented/`
- Microsoft Learn, *Abstract and sealed classes and class members*: `https://learn.microsoft.com/dotnet/csharp/programming-guide/classes-and-structs/abstract-and-sealed-classes-and-class-members`
- Microsoft Learn, *Events*: `https://learn.microsoft.com/dotnet/csharp/programming-guide/events/`
- Microsoft Learn, *Delegates*: `https://learn.microsoft.com/dotnet/csharp/programming-guide/delegates/`
- Microsoft Learn, *Implement a Dispose method*: `https://learn.microsoft.com/dotnet/standard/garbage-collection/implementing-dispose`
- Microsoft Learn, *Windows Forms documentation*: `https://learn.microsoft.com/dotnet/desktop/winforms/`
- Microsoft Learn, *SoundPlayer class*: `https://learn.microsoft.com/dotnet/api/system.media.soundplayer`

Design literature

- Gamma, Helm, Johnson, and Vlissides. *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley, 1994. Observer, Strategy, and Flyweight.
- Martin, Robert C. *Clean Architecture: A Craftsman's Guide to Software Structure and Design*. Prentice Hall, 2017. SOLID principles and dependency direction.
- Fowler, Martin. *Patterns of Enterprise Application Architecture*. Addison-Wesley, 2002. Passive View and Model–View–Presenter.

> **DOCUMENT SCOPE STATEMENT**
>
> This document describes the program as it exists at the stated source baseline. It does not claim that every behavior was exercised interactively; runtime conclusions were derived from source analysis and from the launch and build checks explicitly noted. Page numbers, figures, and file references were generated from that baseline.

End of document · Tindahan ni Aling Nena (Recoded) · Technical Documentation v1.0 · 5 November 2026